

A novel deep convolutional surrogate model with incomplete solve loss for parameterized steady-state diffusion problems

Junqing Jia^a, Xiaoping Zhang^{a,*}, Shuting Wang^b, Lili Ju^b, Hui Feng^a

^a School of Mathematics and Statistics, Wuhan University, Wuhan, 430072, Hubei, China

^b Department of Mathematics, University of South Carolina, Columbia, 29208, SC, USA

ARTICLE INFO

Keywords:

Deep convolutional surrogate model
Parameterized PDEs
Convolutional neural networks
Incomplete solve loss
Picard's iteration

ABSTRACT

In this paper, we propose a deep convolutional surrogate model for learning parameterized steady-state diffusion problems in the end-to-end fashion, which integrates the convolutional neural networks with traditional numerical discretization schemes. The parameters are embedded into images as the input and a slightly modified U-Net is adopted as the backbone of the proposed model. A novel loss function is specially designed for the training process, which incorporates iterative solvers of linear systems and incomplete solves into generation of pseudo data. In addition to linear diffusion problems, our method also can effectively handle nonlinear ones by further utilizing the Picard's iteration with linearization technique. Extensive numerical experiments including ablation studies and applications to engineering problems are presented to verify the effectiveness of the proposed deep convolutional surrogate model and demonstrate its excellent performance in solving parameterized diffusion problems in various scenarios.

1. Introduction

In this paper we consider the parameterized steady-state diffusion problem taking the following form:

$$\begin{cases} -\nabla \cdot (\kappa_\mu \nabla u_\mu) = f_\mu, & \mathbf{x} \in \Omega, \\ u_\mu = g_\mu, & \mathbf{x} \in \Gamma_D, \\ -(\kappa_\mu \nabla u_\mu) \cdot \mathbf{n} = h_\mu, & \mathbf{x} \in \Gamma_N, \end{cases} \quad (1)$$

where $\Omega \subset \mathbb{R}^2$ is an open rectangular domain with boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$, and Γ_D, Γ_N denote the Dirichlet and Neumann boundary, respectively. Here u_μ is the unknown function to be solved, κ_μ is the diffusion coefficient, f_μ, g_μ and h_μ represent the source term parameterized by the parameter μ , the Dirichlet boundary and Neumann boundary values, respectively, and $\mathbf{n}$ denotes the outward unit normal vector on the boundary. Note that if κ_μ depends only on the space variable $\mathbf{x}$, i.e., $\kappa_\mu = \kappa_\mu(\mathbf{x})$, the above problem Eq. (1) is linear, but if κ_μ depends on the solution u_μ , i.e., $\kappa_\mu = \kappa_\mu(u_\mu(\mathbf{x}))$, then it becomes a nonlinear equation.

Parameterized steady-state diffusion equations are central to the understanding of a broad range of phenomena in a variety of physical and engineering disciplines. These equations, a subset of parameterized PDEs, provide insight into the complex behavior of physical quantities or energy as they move through various media. The dynamics and properties of these systems are significantly

* Corresponding author.

E-mail addresses: whujjq@whu.edu.cn (J. Jia), xpzhang.math@whu.edu.cn (X. Zhang), sw156@email.sc.edu (S. Wang), ju@math.sc.edu (L. Ju), hffeng.math@whu.edu.cn (H. Feng).

<https://doi.org/10.1016/j.jcp.2025.114132>

Received 23 September 2024; Received in revised form 12 May 2025; Accepted 27 May 2025

Available online 29 May 2025

0021-9991/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

influenced by the parameters, which include domain shape and size, boundary conditions, material properties, sources, and so on. Understanding and accurately solving parameterized steady diffusion equations is critical for the design and optimization of systems in various fields. Such applications require not only a deep understanding of how the parameters affect the system behavior, but also a real-time solver.

A natural idea for numerically solving parameterized PDEs is to first use traditional numerical methods to repeatedly solve PDEs with a large amount of sampled parameters, and then approximate the solutions corresponding to the expected parameters through some interpolations. This approach consumes huge computational resources and times, especially infeasible for time dependent or high-dimensional problems. Even for low dimensional steady-state problems, it is difficult to meet the real-time requirements of many engineering applications. Therefore, deep convolutional surrogate models emerged, aiming to solve the parameterized PDEs through offline training and online prediction. Although the training stage may require significant computational costs, once the model is well trained, it can predict the solutions of PDEs in real-time given new parameters.

Present deep convolutional surrogate models can be broadly classified into two categories: data-driven and physics-informed. The data-driven method is similar to the supervised learning approach in machine learning and requires a dataset consisting of high-fidelity solutions. Such a dataset is typically obtained using traditional numerical methods with numerous sampled parameters, which is computationally expensive. A deep convolutional surrogate model is then trained to approximate the mapping between the parameter space and the solutions by minimizing the distance between predictions and solutions provided by the dataset. However, a multitude of studies and experiments have demonstrated that although data-driven approaches can provide accurate predictions for parameters included in the dataset, they lack the capacity to generalize solutions for parameters that are not covered in the range of the dataset.

Operator learning is a popular data-driven approach to solving parameterized PDEs that has gained considerable attention. The most notable examples of this approach are the Fourier Neural Operator (FNO) [1] and DeepONet [2], which have demonstrated superior capabilities and efficiency in solving complex problems. The FNO is designed to address the PDEs by transforming the data to the frequency domain through the Fourier transforms, applying a series of learnable Fourier filters, transforming back to the spatial domain, and obtaining the final approximate solutions. The DeepONet is designed to solve the PDEs by learning the mapping between function spaces. This is achieved by a branch network that encodes input functions and a trunk network that predicts target function values. In addition to operator learning, a recent data-driven time-stepping scheme was employed to construct surrogate models [3], effectively evolving the system using a GNN architecture. The incorporation of GNNs enhances the applicability of these surrogate models to complex problems involving arbitrary solution domains and general meshes.

There has been a growing interest in the physics-informed methods for solving PDEs in recent years. Among the most representative works in this field are the Physics Informed Neural Networks (PINNs) [4], the Deep Galerkin Method (DGM) [5] and the Deep Ritz method (DRM) [6]. These methods all employ a fully connected neural network (FCNN) to approximate the solution of PDE directly using spatial and temporal coordinates as input. The primary distinctions between them lie in loss functions. PINN and DGM usually take the summation of residual errors of the PDE and the Mean Square Error (MSE) of the initial/boundary conditions as loss function. The PINN also has been extended to solve inverse problems [4], fractional differential equations [7], stochastic differential equations and uncertainty qualification [8–10] and other applications. DRM [11] designs loss function using the variational form of PDEs, requiring numerical integrations to train the network. Related works have subsequently emerged [12–15]. Although physics-informed methods obviate the necessity for datasets, they are still constrained by substantial training computational expense and low accuracy.

A potential of the physics-informed method is to utilize the expressive capability and generalization ability of neural networks to solve parameterized PDEs, and a series of recent works have focused on this topic. The basic idea behind these works is to pass the PDE's parameters as additional inputs to the neural network, which requires larger network architectures, more sampling data and longer training times. For example, Kovacs et al. [16] developed the conditional PINN to solve a class of eigenvalue problems. The unknown eigenfunction is approximated by a neural network, whose input include not only the points sampled in the problem domain, but also parameters that determine the coefficients of the eigenvalue equation. Zhang et al. [17] extended the proposed GW-PINN to solve pumping wells problem with various pumping capacities in groundwater field, which is helpful for agricultural water management.

In addition to the two paradigms mentioned above, recent works have attempted to integrate the advantages of traditional numerical methods and deep learning techniques to construct deep convolutional surrogate models for solving parameterized PDEs. Fuhg et al. [18] summarized several approaches for training convolution neural networks (CNNs) for parameterized PDEs and finds out that well designed discrete convolution operators and loss functions can significantly improve the model's performance compared to data-driven method. Meanwhile, Zhao et al. [19] and Zhang et al. [20] proposed a CNN-based surrogate model to predict the temperature field, which is modeled by parameterized heat dissipation equations with different source layouts. The aforementioned works were all developed on rectangular meshes in rectangular domains in order to compute the discrete residual loss function with discrete convolution operators effectively. In order to explore the applicability of discrete residual loss functions in general domains, the elliptic coordinate mapping, proposed by [21], transforms the PDE defined on an irregular domain into the identical problem defined on the rectangular domain. Furthermore, Geng et al. [22] not only designed a novel nonlinear residual loss function integrating the finite difference scheme / Crank-Nicolson scheme but also proposed a novel training strategy, which is a great innovation for training the CNN to solve Allen-Cahn equations with different initial conditions.

In this study, we propose an end-to-end deep convolutional surrogate model for the parameterized steady-state diffusion problem Eq. (1), which integrates the convolutional neural networks with traditional numerical discretization schemes. The proposed model embeds parameters into images, feeds them into a slightly modified U-Net, and utilizes a newly designed incomplete solve loss function for the training process. The remainder of the paper is organized as follows. Section 2 provides an overview of traditional numerical

discretization schemes and solvers. Section 3 presents a detailed descriptions to the deep convolutional surrogate model, including the architecture of neural network, loss functions and the training strategies for linear and nonlinear problems. Some ablation studies are conducted in Section 4 to validate the proposed model, and applications to three engineering problems are presented in Section 5 to demonstrate its versatility and robustness in solving various parameterized diffusion problems. Finally, some concluding remarks are given in Section 6.

2. Traditional numerical discretization schemes and solvers

We now introduce some major traditional numerical methods for solving the parameterized diffusion Eq. (1). Finite volume method and finite difference methods for numerical discretization are presented as exemplary approaches, with applicability of other numerical schemes as well. The Picard's approach will be used for iterative solution of the resulting nonlinear system if the problem is nonlinear. The traditional numerical discretization and solvers will be used later to construct loss functions for training our deep convolutional surrogate model in various scenarios. For convenience, we assume the domain $\Omega = (a, b) \times (c, d)$ and omit the parameter μ from the Eq. (1) in the following discussions.

2.1. Discretization schemes

In this subsection, we assume that the problem Eq. (1) is linear, i.e., $\kappa_\mu = \kappa_\mu(\mathbf{x})$, and present two numerical schemes on a uniform rectangular mesh: the vertex-centered finite difference (FD) scheme and the cell-centered finite volume (FV) scheme.

2.1.0.1. The vertex-centered FD scheme. Given a vertex-centered uniform rectangular mesh $\mathcal{M}_v$ of size $N_x \times N_y$ defined by

$$\mathcal{M}_v = \{\mathbf{x}_{ij} = (x_i, y_j) \mid x_i = a + ih_x, i = 0, \dots, N_x - 1, y_j = c + jh_y, j = 0, \dots, N_y - 1\}$$

with $h_x = (b - a)/(N_x - 1)$ and $h_y = (d - c)/(N_y - 1)$, the standard vertex-centered FD scheme for solving Eq. (1) can be described as

$$-\sum_{i' \in \{-1, 1\}} \omega_{i+i', j} (u_{i+i', j} - u_{ij}) - \sum_{j' \in \{-1, 1\}} \omega_{i, j+j'} (u_{i, j+j'} - u_{ij}) = f_{ij} h_x h_y, \quad (2)$$

where u_{ij} is the FD approximation of $u(\mathbf{x}_{ij})$, $f_{ij} = f(\mathbf{x}_{ij})$, $\kappa_{ij} = \kappa(\mathbf{x}_{ij})$, and

$$\omega_{i+i', j} = m_A(\kappa_{ij}, \kappa_{i+i', j}), \quad \omega_{i, j+j'} = m_A(\kappa_{ij}, \kappa_{i, j+j'}).$$

Here, $m_A(\alpha, \beta)$ denotes the arithmetic mean of α and β , i.e., $m_A(\alpha, \beta) = \frac{\alpha + \beta}{2}$.

2.1.0.2. The cell-centered FV scheme. Given a cell-centered uniform rectangular mesh $\mathcal{M}_c$ of size $N_x \times N_y$ defined by

$$\mathcal{M}_c = \{\bar{\mathbf{x}}_{ij} = (\bar{x}_i, \bar{y}_j) \mid \bar{x}_i = a + (i - \frac{1}{2})h_x, i = 1, \dots, N_x, \bar{y}_j = c + (j - \frac{1}{2})h_y, j = 1, \dots, N_y\}$$

with $h_x = (b - a)/N_x$ and $h_y = (d - c)/N_y$, a cell-centered FV scheme derived from [23] for solving Eq. (1) can be described as

$$-\sum_{i' \in \{-1, 1\}} \bar{\omega}_{i+i', j} (\bar{u}_{i+i', j} - \bar{u}_{ij}) - \sum_{j' \in \{-1, 1\}} \bar{\omega}_{i, j+j'} (\bar{u}_{i, j+j'} - \bar{u}_{ij}) = \bar{f}_{ij} h_x h_y, \quad (3)$$

where $\bar{u}_{ij}$ is the FV approximation of $u(\bar{\mathbf{x}}_{ij})$, $\bar{f}_{ij} = f(\bar{\mathbf{x}}_{ij})$, $\bar{\kappa}_{ij} = \kappa(\bar{\mathbf{x}}_{ij})$, and

$$\bar{\omega}_{i+i', j} = m_H(\bar{\kappa}_{ij}, \bar{\kappa}_{i+i', j}), \quad \bar{\omega}_{i, j+j'} = m_H(\bar{\kappa}_{ij}, \bar{\kappa}_{i, j+j'}).$$

Here, $m_H(\alpha, \beta)$ denotes the harmonic mean of α and β , i.e., $m_H(\alpha, \beta) = \frac{2}{\alpha^{-1} + \beta^{-1}}$.

The resulting linear system from Eqs. (2) or (3) can be efficiently solved by using many existing linear system solvers [24]. The cell-centered FV scheme Eq. (3) on the rectangular mesh is a specific case of the interpolation-free cell-centered finite volume scheme proposed by [23]. The latter is applicable for handling anisotropic, heterogeneous, and discontinuous diffusion tensors, achieving second-order accuracy on arbitrary polygonal meshes. The vertex-centered FD scheme Eq. (2) achieves second-order accuracy only for the diffusion problems with continuous diffusive coefficients, whereas the cell-centered FV scheme Eq. (3) maintains second-order accuracy even for the diffusion problems with discontinuous diffusive coefficients. Additionally, it is worth noting that the vertex-centered FD scheme is well-suited for handling Dirichlet boundary conditions, whereas the cell-centered FV scheme can be more convenient when Neumann boundary conditions are also involved.

2.2. Picard's iterative method for nonlinear system

If the diffusion problem Eq. (1) is nonlinear, i.e., $\kappa_\mu = \kappa_\mu(u_\mu(\mathbf{x}))$, we can apply the Picard's iterative approach to solve it in combination with the discretization schemes presented in Section 2.1. Starting from a given initial guess u^0 , the Picard's method iteratively solves a series of linear problems to approximate the solution, where the diffusive coefficient in Eq. (1) is linearized within each step of the Picard's iterations. Specifically, in the $(p + 1)$ -th iteration, the following linearized equation needs to be solved

$$\begin{cases} -\nabla \cdot \kappa(u^p(\mathbf{x})) \nabla u(\mathbf{x}) = f(\mathbf{x}), & \mathbf{x} \in \Omega, \\ u(\mathbf{x}) = g_D(\mathbf{x}), & \mathbf{x} \in \Gamma_D, \\ -(\kappa(u^p(\mathbf{x})) \nabla u(\mathbf{x})) \cdot \mathbf{n} = g_N(\mathbf{x}), & \mathbf{x} \in \Gamma_N, \end{cases} \quad (4)$$

where u^p denotes the solution of p -th Picard's iteration and then set $u^{p+1} = u$. The aforementioned finite volume and finite difference scheme are both applicable to solve Eq. (4) of $p = 0, 1, 2, \dots$ until the L_2 error between solutions of adjacent Picard's steps is less than a predefined threshold ε^p . The detailed computation flow of Picard's method is presented in Algorithm 1.

Algorithm 1: Picard's iteration for solving the nonlinear diffusion problem.

Input: Initial solution u^0 , threshold ε^p , maximum number of iterations $P_{\max}$

```

1 for  $p = 0, 1, 2, \dots, P_{\max}$  do
2   Solve the linearized equation (4) by FVM or FDM;
3   if  $\|u^p - u\| \leq \varepsilon^p$  then
4     BREAK;
5    $u^{p+1} \leftarrow u$ ;

```

Result: u

3. The proposed deep convolutional surrogate model

In this section we introduce the major components of our deep convolutional surrogate model for parameterized diffusion problems, including the architecture of neural network, loss functions and training strategies.

3.1. The architecture of neural network

The architecture of neural networks has undergone a significant evolution over the years. Initially, convolutional neural networks (CNNs) were primarily used for image classification tasks, and later researchers have introduced many variations, expanding their applications to other computer vision tasks, such as semantic segmentation and depth prediction of images. Among the most influential architectures are ResNet [25] and U-Net [26]. The CNN architecture proposed in our method is a slight modification of U-Net, which accepts the parameter μ in the format of an image as input.

Let us begin with introducing two fundamental components of our CNN architecture: the residual block and the convolutional block, which are illustrated at the top of Fig. 1. Both blocks consist of two convolutional layers, maintaining the same number of input and output channels. Each convolutional layer is followed by a normalization and an activation function, which are standard practices for promoting stability and non-linearity. Now we explain the proposed CNN architecture depicted at the bottom of Fig. 1. This architecture is structured into L levels (an example of $L = 4$ is illustrated in Fig. 1). Slightly different from standard U-Nets, our network starts with an input header $\mathcal{H}$, which is used to extract features, adjust the number of channels, and preserve the spatial dimensions of the input with a convolutional block. Subsequently, the encoder comprises L submodules, denoted by $\{\mathcal{E}_l\}_{l=1}^L$, each is designed to extract features at different resolution levels. Every submodule $\mathcal{E}_l$ in the encoder is composed of N_l stacked residual blocks followed by a max pooling operation to reduce the spatial dimensions. The extracted features are then fed into the decoder which also has L submodules $\{\mathcal{D}_l\}_{l=1}^L$. Every decoder submodule $\mathcal{D}_l$ also comprises N_l stacked residual blocks paired with bilinear interpolation for upsampling and is used to gradually restore the spatial size for matching the original input. The architecture concludes with an additional output tail $\mathcal{T}$, which employs a 1×1 convolution without an activation function to obtain the final prediction. Additionally, the bottleneck $\mathcal{B}$ utilizes a convolutional block to facilitate skip connections in the last level, enhancing feature integration across the network. The proposed CNN architecture takes advantage of both U-Net and ResNet, which are designed for being expanded easily and trained robustly.

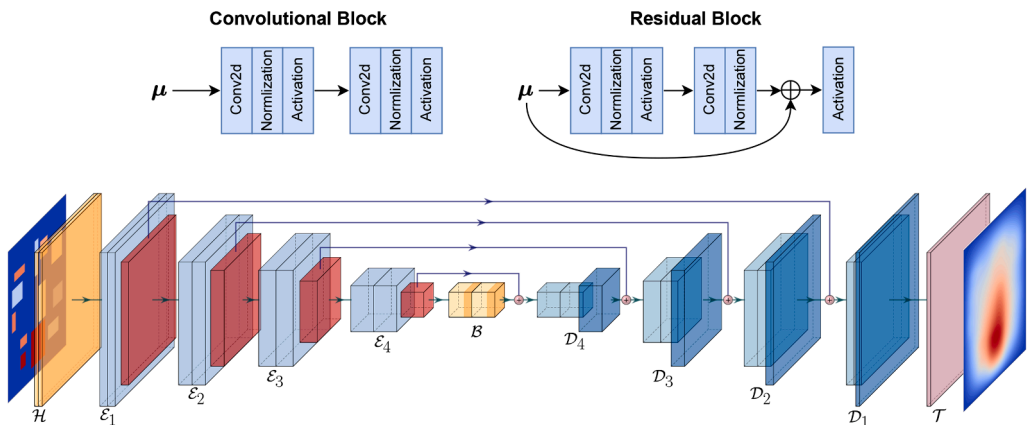


Fig. 1. The network architecture based on U-Net for our proposed deep convolutional surrogate model.

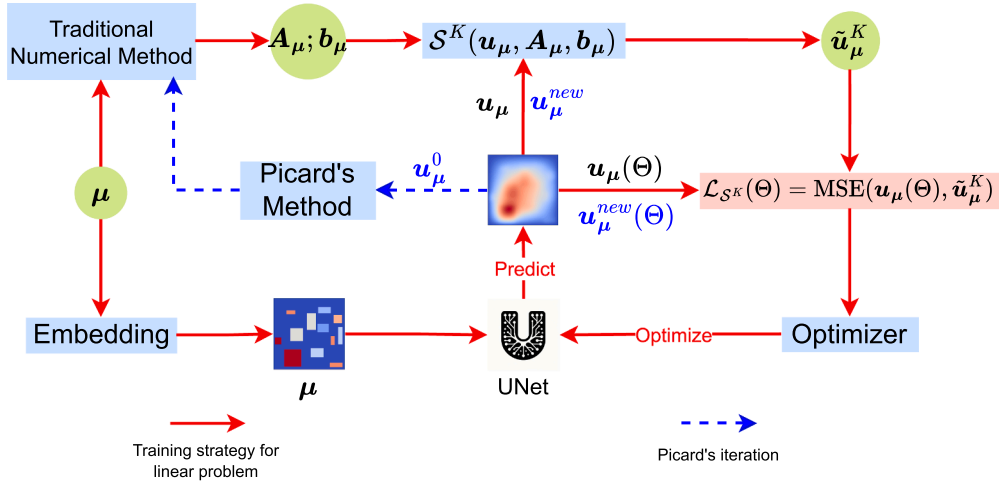


Fig. 2. The flowchart of the training strategy with loss function for our deep convolutional surrogate model. The dashed blue lines represent Picard's iteration need for nonlinear diffusion problems, where the prediction is used to linearize the nonlinear diffusion problem and the other lines indicates the standard training strategy. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Next let us delve deeper into the specifics of this architecture. Assume the input μ has the size of $B \times C_{in} \times H \times W$, where B is the batch size, $H \times W$ represents the spatial size, and C_{in} is determined by the specific embedding methods. The input header $\mathcal{H}$ processes μ to extract features with dimension $B \times P \times H \times W$, where P is typically chosen to be greater than C_{in} to enhance feature capacity. Each encoder module $\mathcal{E}_i$ takes the output from $\mathcal{H}$ or preceding encoder module $\mathcal{E}_{i-1}$ as its input. The last residual block in $\mathcal{E}_i$ is followed by the max pooling operation and output features with a spatial resolution that is half of its input and a channel count that is twice of its input. The remaining residual blocks in $\mathcal{E}_i$ focus on feature extraction at the original spatial resolution. The decoder modules $\mathcal{D}_i$ then integrate the features by summing the outputs of $\mathcal{E}_i$ and $\mathcal{D}_{i+1}$ or the bottleneck $\mathcal{B}$. The last residual block in $\mathcal{D}_i$, followed by the bilinear interpolation, operates the features by doubling its spatial size and halving the number of channels, effectively rebuilding the resolution towards that of the original input. The rest of residual blocks in $\mathcal{D}_i$ focus on extracting features in the coarser resolution. Finally, the output tail $\mathcal{T}$ processes the output from $\mathcal{D}_1$ to produce the final prediction $u_\mu(\Theta)$ with shape $B \times 1 \times H \times W$. This component ensures that the features decoded from the network are translated back into a coherent prediction, matching the intended output resolution and detail.

Based on the above introduction, it is easy to see that the proposed network architecture of our deep convolutional surrogate model is slightly modified from U-Net and its performance depends on the specified parameters P , L , and $\{N_l\}_{l=1}^L$, abbreviated as $\text{U-Net}(P, L, (N_1, \dots, N_L))$.

3.2. The loss functions and training strategies

In our method, the parameter μ is first embedded into an image format using the uniform rectangular mesh before being fed into the modified U-Net discussed in the preceding section. The embedding method is designed based on the problem settings and some examples will be introduced later through numerical experiments. A flowchart of the training strategy with loss function for our deep convolutional surrogate model is given in Fig. 2 and we will illustrate it in detail in the following.

3.2.1. On parameterized linear diffusion problems

It is well known that traditional numerical methods often result in large-scale sparse linear systems, which are usually solved by iterative linear solvers [24], such as Jacobi iteration, conjugate gradient method, GMRES, and others. Let us denote by $S^K(x_0, \mathbf{A}, \mathbf{b})$ an iterative solver for the linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$, where x_0 is an initial guess and K is the maximum number of iterations. The iterative solver is also the key to integrating traditional numerical methods with deep learning techniques in our method.

After sampling a parameter μ from the parameter space $\mathcal{P}_\mu$, the parameterized linear diffusion problem Eq. (1) is uniquely determined. Then we use the traditional numerical schemes discussed in Section 2.1 to assemble the stiffness matrix $\mathbf{A}_\mu$ and the load vector $\mathbf{b}_\mu$. At the same time, the image format of μ is fed into the U-Net and the corresponding output $u_\mu(\Theta)$ is then taken as the initial guess for the iterative linear system solver. Based on $\mathbf{A}_\mu$, $\mathbf{b}_\mu$ and $u_\mu(\Theta)$, a pseudo-labeled data is obtained through iterative solvers, i.e.,

$$\tilde{u}_\mu^K = S^K(u_\mu(\Theta), \mathbf{A}_\mu, \mathbf{b}_\mu). \quad (5)$$

Then, the mean square error (MSE) is utilized as the loss function to quantify the difference between the output prediction $u_\mu(\Theta)$ and the pseudo-labeled data $\tilde{u}_\mu^K$, i.e.,

$$\mathcal{L}_{S^K}(\Theta) = \text{MSE}(u_\mu(\Theta), \tilde{u}_\mu^K), \quad (6)$$

where

$$\text{MSE}(\mathbf{u}, \mathbf{v}) = \frac{1}{N_x \times N_y} \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} (u_{ij} - v_{ij})^2$$

for any two $N_x \times N_y$ images $\mathbf{u}$ and $\mathbf{v}$. We will call the loss function defined by Eq. (6) as the *incomplete solve loss*. Note that we also detach the gradient of $\tilde{\mathbf{u}}_\mu^K$ and only execute the back-propagation through $\mathbf{u}_\mu(\Theta)$. This is crucial in training; otherwise the model could be difficult or impossible to train.

Given a traditional numerical method, we introduce two linear iterative solvers for constructing the *incomplete solve loss* functions. The first one is the Jacobi iteration method which can be formulated as follows: starting with $\mathbf{u}_0$, perform the iteration

$$\mathbf{u}_{i+1} = \mathbf{D}^{-1}(\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{u}_i), \quad i = 0, 1, \dots, \quad (7)$$

where $\mathbf{D}$, $\mathbf{L}$, and $\mathbf{U}$ denote the diagonal, lower triangular and upper triangular parts of $\mathbf{A}$, respectively. The other one uses the steepest descent method which can be formulated as follows: starting with $\mathbf{u}_0$, perform the iteration

$$\mathbf{r}_i = \mathbf{b} - \mathbf{A}\mathbf{u}_i, \quad \alpha_i = \frac{\mathbf{r}_i^T \mathbf{r}_i}{\mathbf{r}_i^T \mathbf{A} \mathbf{r}_i}, \quad \mathbf{u}_{i+1} = \mathbf{u}_i + \alpha_i \mathbf{r}_i, \quad i = 0, 1, \dots. \quad (8)$$

The Jacobi iteration method Eq. (7) and the steepest descent method Eq. (8) with a maximum of K iterations are denoted as $\mathcal{J}^K(\mathbf{u}_0, \mathbf{A}, \mathbf{b})$ and $\mathcal{D}^K(\mathbf{u}_0, \mathbf{A}, \mathbf{b})$, respectively. Consequently, for the proposed incomplete solve loss function Eq. (6), we let $\mathcal{L}_{\mathcal{J}^K}(\Theta)$ denote the loss using the Jacobi iterative solver and $\mathcal{L}_{\mathcal{D}^K}(\Theta)$ represent the one using the steepest descent solver. It is important to emphasize that the proposed incomplete solve loss can also be constructed using other linear iterative solvers.

Remark 1. There are two other commonly used loss functions. One is the *discrete residual loss*, which is derived directly from the linear system $\mathbf{A}\mathbf{u} = \mathbf{b}$ and can be formulated as:

$$\mathcal{L}_{\text{res}}(\Theta) = \text{MSE}(\mathbf{A}\mathbf{u}(\Theta) - \mathbf{b}, \mathbf{0}). \quad (9)$$

The other is the *data loss*, which is based on high-fidelity solutions denoted by $\hat{\mathbf{u}}$, i.e.,

$$\mathcal{L}_{\text{data}}(\Theta) = \text{MSE}(\mathbf{u}(\Theta), \hat{\mathbf{u}}). \quad (10)$$

The motivation behind our proposed incomplete solve loss Eq. (6) is that, if an iterative linear solver converges, it will produce a more accurate solution after a finite number of iterations. In other words, if K is sufficiently large, the iterative solver will generate highly accurate labeled data, thereby transforming the deep convolutional surrogate model into a data-driven paradigm. In practice, K is typically selected as a relatively modest value which may be quite small. It is important to point out that the role of the iterative linear solver in this context is to guide the network, rather than to completely solve linear equations.

With the help of loss function $\mathcal{L}_{\mathcal{S}^K}(\Theta)$, our deep convolutional surrogate model for solving a parameterized linear diffusion problem can be trained in a standard approach. To clearly illustrate the training process, we provide a training flowchart in Fig. 2 (ignoring Picard's iterations) and the corresponding algorithm in Algorithm 2. In the initial stages, the deep convolutional surrogate model is not yet fully trained, resulting in lower prediction accuracy. Therefore, the pseudo-labeled data $\tilde{\mathbf{u}}_\mu^K$ generated from the iterative solver $\mathcal{S}^K$ may not be as accurate as desired. However, as training progresses, the accuracy of deep convolutional surrogate model is expected to be improved, allowing it to predict better initial guess $\mathbf{u}_\mu(\Theta)$ for the linear iterative solver. This improvement enables $\mathcal{S}^K$ to provide more accurate pseudo-labeled data.

Algorithm 2: The training strategy of a parameterized linear problem.

Data: The parameter space $\mathcal{P}_\mu$

Input: The maximum number of epochs E_{max} , the maximum number of iterations K for iterative linear solver

```

1 for epoch = 1, 2, ..., E_max do
2   for  $\mu \in \mathcal{P}_\mu$  do
3     Embed  $\mu$  into image format, still denoted by  $\mu$ ;
4     Feed  $\mu$  into the U-Net to make a prediction  $\mathbf{u}_\mu(\Theta)$ ;
5     Assemble the stiffness matrix  $\mathbf{A}_\mu$  and the load vector  $\mathbf{b}_\mu$ ;
6     Using the iterative solver (5) to obtain  $\tilde{\mathbf{u}}_\mu^K = \mathcal{S}^K(\mathbf{u}_\mu(\Theta), \mathbf{A}_\mu, \mathbf{b}_\mu)$ ;
7     Compute the incomplete solve loss  $\mathcal{L}_{\mathcal{S}^K}(\Theta) = \|\mathbf{u}_\mu(\Theta) - \tilde{\mathbf{u}}_\mu^K\|^2$ , where the gradient of  $\tilde{\mathbf{u}}_\mu^K$  is detached;
8     Update  $\Theta$  by Adam optimizer;
```

Result: Θ

3.2.2. On parameterized nonlinear diffusion problems

Due to the inherent nonlinearity of nonlinear diffusion problems, the aforementioned training strategy cannot be directly applied and some modifications are required. Drawing inspiration from Picard's method for solving nonlinear systems reviewed in Section 2.2, the training strategy for parameterized nonlinear diffusion problems are designed by extending the standard training strategy described in Algorithm 2, in which Picard's iteration is particularly nested into the standard strategy. For details, we refer to Algorithm 3 and Fig. 2 with the dashed blue lines included.

Algorithm 3: Training strategy for solving nonlinear problem.

Data: The parameter space $\mathcal{P}_\mu$
Input: The maximum number of epochs $E_{\max}$, the maximum number of iterations K for the iterative linear solver, the initial guess u^0 , the maximum Picard's iteration number $P_{\max}$ and the threshold ε^p for Picard's iteration.

```

1 for epoch = 1, 2, ...,  $E_{\max}$  do
2   for  $\mu \in \mathcal{P}_\mu$  do
3     Embed  $\mu$  into image format, still denoted by  $\mu$ ;
4     if epoch = 1 then
5        $u_\mu^0 = u^0$ ;
6     else
7       Feed  $\mu$  into the U-Net to make a prediction  $u_\mu = u_\mu(\Theta)$ ;
8        $u_\mu^0 = u_\mu$ ;
9     for  $p = 1, \dots, P_{\max}$  do
10      Assemble the stiffness matrix  $A_\mu$  and the load vector  $b_\mu$  based on  $u_\mu^0$ ;
11      Feed  $\mu$  into the U-Net to make a prediction  $u_\mu^{new} = u_\mu^{new}(\Theta)$ ;
12      Using the iterative solver (5) to obtain  $\tilde{u}_\mu^K = S^K(u_\mu^{new}, A_\mu, b_\mu)$ ;
13      Compute the incomplete solve loss  $\mathcal{L}_{SK}(\Theta) = \text{MSE}(u_\mu^{new}(\Theta), \tilde{u}_\mu^K)$ ;
14      Update  $\Theta$  by optimizer;
15      Compute the error between two successive steps in Picard's iteration  $e = \|u_\mu^{new} - u_\mu^0\|_{L_2}$ ;
16      if  $e < \varepsilon^p$  then
17        BREAK;
18      else
19         $u_\mu^0 \leftarrow u_\mu^{new}$ ;

```

Result: Θ

As presented in Algorithm 3, the loop on μ which traverses the parameter space $\mathcal{P}_\mu$ to guarantee that the deep convolutional surrogate model can adapt to different parameters μ . A sampled parameter μ corresponds to a specific nonlinear equation whose solution will be approximated by the U-Net trained throughout the Picard's iteration. Same like the maximum number of iterations K for the iterative linear solver, the maximum Picard's iteration number $P_{\max}$ is typically chosen small such that the U-Net will not be optimized on a specific μ for too many steps, therefore avoiding overfitted on the corresponding nonlinear equation. That is to say, the training strategy for nonlinear problems is based on the incomplete Picard's iteration, which shares the similar idea as the incomplete solve loss based on a iterative linear solver S^K .

Within each of the Picard's loop, the U-Net is trained to approximate the solution of the corresponding linearized problem Eq. (4). Typically, a pre-specified good initial value is often selected for the traditional Picard's method in order to robustly solve the nonlinear problem. Consistent with the traditional Picard's approach, a manually selected initial value u^0 will also be provided at the first epoch to achieve a stable training process. In the subsequent epochs, the predictions from the U-Net are used as the initial values for Picard's iterations, as the network has been trained to a certain extent and is able to provide more accurate initial values. The Picard's loop is monitored by the error between the predictions of two consecutive Picard's steps; when the error drops below the predefined threshold ε^p , the Picard's loop will be terminated.

4. Ablation studies

In this section, we will perform some ablation studies on the proposed deep convolutional surrogate model using three simplified PDE problems defined on the rectangular domain. The purpose of the ablation study is to systematically analyze how different hyperparameters-such as grid resolution, model architecture, and loss function-affect the performance of the surrogate model. By quantifying the numerical results under various hyperparameter settings, the study provides practical guidance for selecting suitable hyperparameters when applying the model to other engineering problems. Specifically, the first test case involves a non-parameterized Poisson equation, which is used to examine the potential impact of different loss functions, activation functions, and normalization methods on model performance. The second case considers a parameterized diffusion equation, aiming to investigate how the number

of iterations K in the iterative linear solver and the number of parameter samples μ influence prediction accuracy. The third case involves a 3D parameterized Poisson equation, which serves to evaluate the scalability and adaptability of the deep convolutional surrogate model in more complex, higher-dimensional problems.

4.1. 2D parameter-free Poisson equation

Let us consider the parameter-free Poisson equation defined on the domain $\Omega = (-1, 1)^2$ with homogeneous Dirichlet boundary condition

$$\begin{cases} -\Delta u(\mathbf{x}) = f(\mathbf{x}), & \mathbf{x} \in \Omega, \\ u(\mathbf{x}) = 0, & \mathbf{x} \in \partial\Omega, \end{cases} \quad (11)$$

where the source term is chosen as $f(\mathbf{x}) = 2\lambda^2\pi^2 \sin(\lambda\pi x_1) \sin(\lambda\pi x_2)$ and thus the exact solution is given by $u(\mathbf{x}) = \sin(\lambda\pi x_1) \sin(\lambda\pi x_2)$. Note here λ is not a parameter and will be given in advance in each case. Since there are no additional parameters, we set the batch size to be 1, and choose the input of the U-Net to be the vertex coordinates of the mesh $\mathcal{M}_v$, which forms a two-channel image of shape $2 \times N_x \times N_y$. Note that other choices for the input are also applicable, for instances, the source f in the form of an image of shape $N_x \times N_y$.

The vertex-centered FD scheme Eq. (2) is used to discretize Eq. (11) with $N_x = N_y = 256$ (i.e., $h_x = h_y = 2/255$). In this case, we obtain the linear discrete system as

$$-u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1} + 4u_{ij} = h^2 f_{ij}, \quad i, j = 1, \dots, N-1, \quad (12)$$

where the $u_{ij} = u(\mathbf{x}_{ij})$ and $f_{ij} = f(\mathbf{x}_{ij})$. The matrix form of this scheme is denoted as $\mathbf{A}\mathbf{u} = \mathbf{b}$.

4.1.1. Effect of loss functions

We compare the performance of the proposed deep convolutional surrogate model under different types of loss functions for solving the Eq. (11), including the incomplete solve loss functions $\mathcal{L}_{JK}(\Theta)$ and $\mathcal{L}_{DK}(\Theta)$, defined in the subsection Section 3.2.1, as well as the discrete residual loss function $\mathcal{L}_{\text{res}}$, defined in Eq. (9), and the data loss $\mathcal{L}_{\text{data}}$, defined in Eq. (10). The U-Net(8, 4, (2, 2, 2, 2)) with layer normalization and tanh activation function is used as the backbone and optimized by the Adam optimizer with Algorithm 2. There are 150 epochs in the entire training process and the U-Net is optimized for 100 steps within each epoch. An exponential decaying learning rate whose initial learning rate is set to 0.01 is used to adjust the learning rate for each epoch. Solutions of the

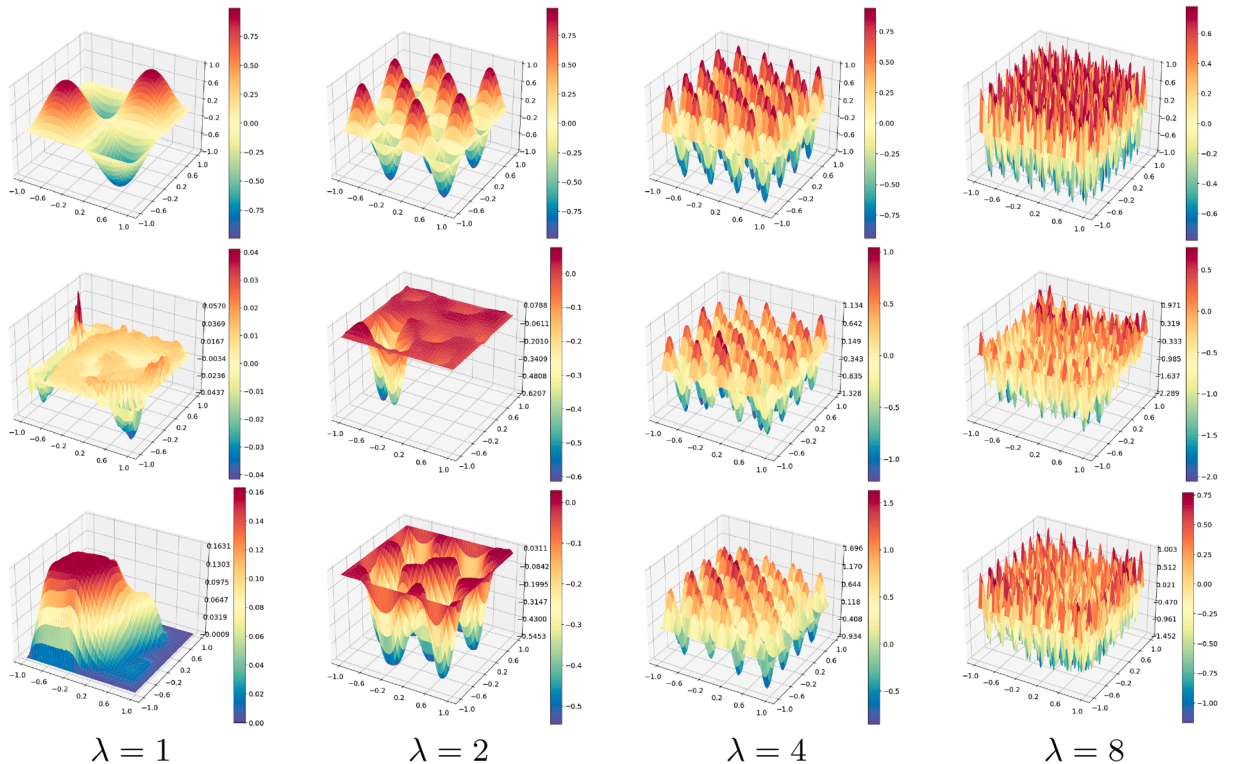


Fig. 3. The predictions for the Poisson Eq. (11) produced by the proposed deep convolutional surrogate model trained with the discrete residual loss function $\mathcal{L}_{\text{res}}(\Theta)$. Top row: the reference solutions, second and third rows: the results using ReLU and tanh activations, respectively.

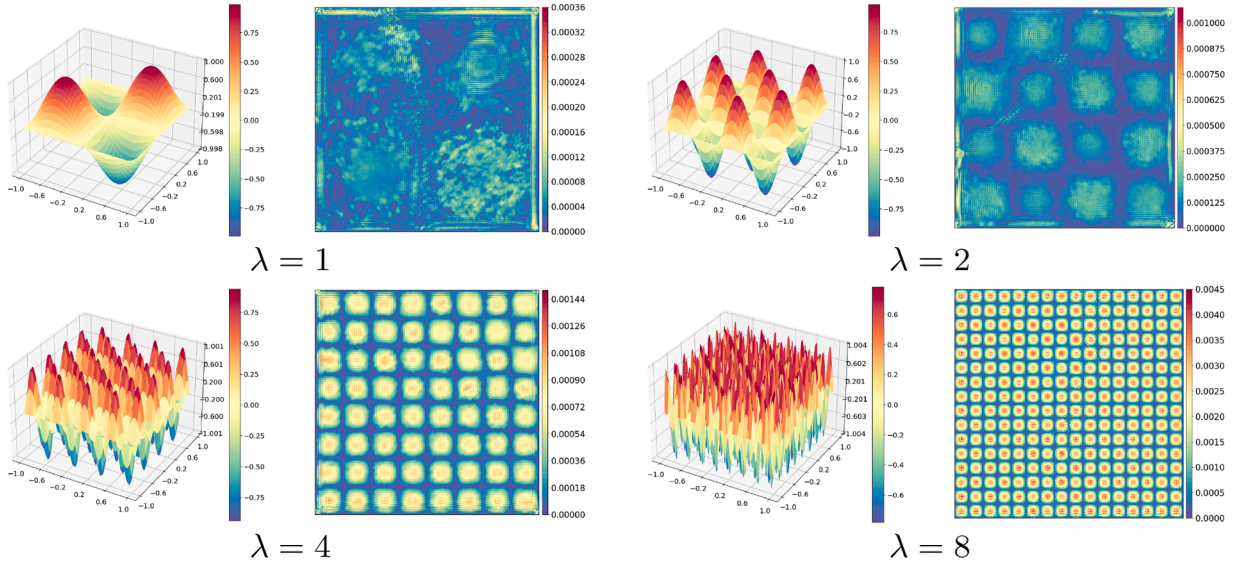


Fig. 4. The predictions for the Poisson Eq. (11) produced by the proposed deep convolutional surrogate model utilizing the tanh activation function and trained with the incomplete solve loss $\mathcal{L}_{J^3}$.

problem Eq. (11) with four settings $\lambda = [1, 2, 4, 8]$ are learned separately. Clearly the solution frequency doubles repeatedly on both x and y directions along the doubling of the value of λ .

We first set $K = 3$ and Fig. 3 presents the predictions from the proposed deep convolutional surrogate model trained with the discrete residual loss function $\mathcal{L}_{\text{res}}(\Theta)$. It can be observed that regardless of the activation function employed, the $\mathcal{L}_{\text{res}}(\Theta)$ is unable to effectively train the deep convolutional surrogate model, resulting in predictions that are significantly different from the reference solutions for all values of λ . We think this is partly because the discrete residual loss $\mathcal{L}_{\text{res}}(\Theta)$ directly contains the linear system information and the large condition number of $\mathbf{A}$ makes the learning to be sensitive and hard. In contrast, Fig. 4 shows the predictions of the proposed deep convolutional surrogate model utilizing the tanh activation function and trained with the incomplete solve loss $\mathcal{L}_{J^3}$. The results demonstrate that the deep convolutional surrogate model performs exceptionally well, even for high-frequency solutions ($\lambda = 8$).

We report in Fig. 5 evolutions of the L_2 solution errors produced by the deep convolutional surrogate model trained with different loss functions during the whole training process. It can be observed that for low-frequency problems ($\lambda = 1, 2, 4$), the deep convolutional surrogate models trained with the incomplete solve losses $\mathcal{L}_{J^3}$ and $\mathcal{L}_{D^3}$ both achieve better performance than that with the

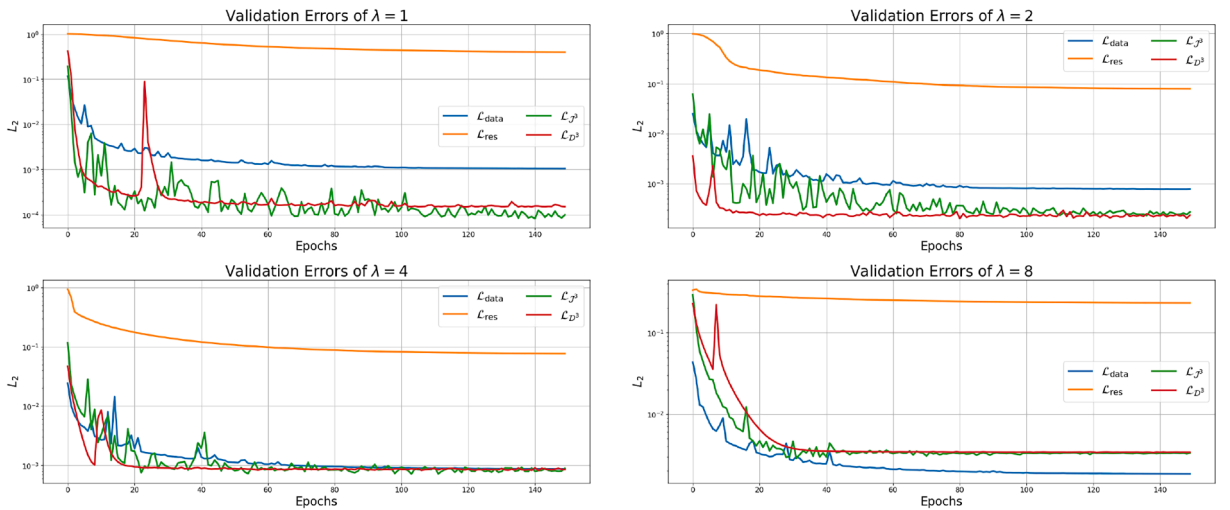


Fig. 5. Evolutions of the L_2 solution errors during the whole training process for the Poisson Eq. (11) produced by the deep convolutional surrogate model trained with different loss functions.

Table 1

Numerical results for ablation study on the effect of activation functions and normalization methods.

Loss	Normalization	Activation	L^2 solution errors			
			$\lambda = 1$	$\lambda = 2$	$\lambda = 4$	$\lambda = 8$
$\mathcal{L}_{J^3}$	Batch	ReLU	4.89e-2	1.70e-2	1.26e-2	8.96e-2
		tanh	1.77e-3	3.52e-3	3.91e-3	9.17e-3
	Layer	ReLU	1.84e-4	3.37e-4	1.08e-3	3.40e-3
		tanh	9.84e-5	2.72e-4	8.98e-4	3.34e-3
$\mathcal{L}_{D^3}$	Batch	ReLU	1.24e-2	3.90e-2	1.06e-2	9.51e-3
		tanh	1.84e-3	3.11e-3	4.25e-3	1.65e-2
	Layer	ReLU	2.16e-4	2.90e-4	9.06e-4	3.27e-3
		tanh	1.50e-4	2.35e-4	8.61e-4	3.40e-3

data loss $\mathcal{L}_{\text{data}}$. Although the data-driven method performs the best on the high-frequency problem ($\lambda = 8$), the approaches with the incomplete solve loss functions also obtain comparable results. In all cases, the deep convolutional surrogate model trained with the residual loss $\mathcal{L}_{\text{res}}$ remains high L_2 errors throughout the training process, indicating its inefficiency and bad performance. Therefore, in subsequent experiments, we will no longer consider the discrete residual loss $\mathcal{L}_{\text{res}}$.

Remark 2. The data loss $\mathcal{L}_{\text{data}}$ is employed for training the deep convolutional surrogate model in a supervised manner, thus it necessitates pre-generating data using high-precision numerical solvers, which incurs considerable computational costs overheads. Moreover, the supervised methods frequently constrain the model's generalization ability. The discrete residual loss $\mathcal{L}_{\text{data}}$ resembles the loss used in PINNs, and it is employed to train the deep convolutional surrogate model in an unsupervised manner. While simple and intuitive, subsequent numerical experiments have revealed that training the deep convolutional surrogate model using the residual loss often does not yield accurate predictions. This observation motivates us to seek a more effective and more versatile loss function, which forms the cornerstone of our work.

4.1.2. Effect of activation functions and normalization methods

The ablation study on the effect of activation functions and normalization methods is discussed with the same Poisson Eq. (11). The detailed quantitative experimental results can be found in Table 1. Extensive works have shown that the neural networks using tanh activation function is preferable to those using ReLU on solving the PDE, since solving PDE with deep learning techniques is a regression problem rather than a classification mission. The same conclusion are also observed from the mean L_2 solution errors in the Table 1, revealing the following insights: 1) Across most scenarios, regardless of the normalization method or iterative solver used, the tanh activation function generally outperforms the ReLU activation function in solving PDEs, and their performance is nearly equal for the problem of high-frequency solutions. 2) In all tested cases, layer normalization significantly outperforms batch normalization.

4.1.3. Effect of mesh resolutions

We further investigate the effect of mesh resolutions on the performance of the deep convolutional surrogate model for solving the parameter-free PDEs. To this end, we consider four different mesh resolutions ($N_x = N_y = 32, 128, 256, 512$), using a U-Net(8, 4, (2, 2, 2, 2)) with layer normalization and tanh activation function as the backbone network. The model is trained using the incomplete solve loss function $\mathcal{L}_{J^3}$. The corresponding numerical results are reported in Table 2. As the mesh becomes finer, we observe a clear decrease in the L_2 solution error. Notably, the convergence rate is approximately 2 for all considered values of $\lambda = 1, 2, 4, 8$, which is consistent with the theoretical convergence rate of traditional numerical schemes. This demonstrates that high-resolution meshes are beneficial for enhancing the predictive performance of the deep convolutional surrogate model.

Table 2

Numerical performance of the deep convolutional surrogate model for solving the parameter-free PDE at different mesh resolutions.

	$N_x = N_y$	32	64	128	256	512
$\lambda = 1$	L_2 error	3.43e-3	8.45e-4	2.26e-4	9.04e-5	3.05e-5
	rate	-	2.0	1.9	1.3	1.5
$\lambda = 2$	L_2 error	1.38e-2	3.33e-3	8.26e-4	2.22e-4	6.38e-5
	rate	-	2.04	2.01	1.89	1.79
$\lambda = 4$	L_2 error	5.66e-2	1.33e-2	3.28e-3	8.30e-4	2.07e-4
	rate	-	2.08	2.02	1.98	1.99
$\lambda = 8$	L_2 error	2.51e-1	5.47e-2	1.31e-2	3.27e-3	8.14e-4
	rate	-	2.19	2.05	2.00	2.00

Table 3

The mean L_2 solution errors produced by the proposed deep convolutional surrogate model with different values of K and N_μ .

$N_\mu \backslash K$	$K = 1$		$K = 5$		$K = 25$	
	$\mathcal{L}_{J^1}$	$\mathcal{L}_{D^1}$	$\mathcal{L}_{J^5}$	$\mathcal{L}_{D^5}$	$\mathcal{L}_{J^{25}}$	$\mathcal{L}_{D^{25}}$
100	9.21e-2	8.68e-2	2.83e-2	6.50e-2	2.12e-2	2.04e-2
500	7.93e-3	7.50e-3	1.61e-3	3.49e-3	1.26e-3	1.78e-3
5000	2.66e-3	4.52e-3	2.76e-3	1.97e-3	7.29e-4	6.67e-4

4.2. 2D parameterized PDEs

Although the effectiveness of $\mathcal{L}_{SK}$ in training the proposed deep convolutional surrogate model to solve a parameter-free PDE has been validated, further testing is needed for parameterized PDEs.

4.2.1. Effect of the iteration number of the incomplete solve loss and the number of training sample

To investigate how the iteration number K of the incomplete solve loss $\mathcal{L}_{SK}$ and the number of training samples N_μ affect the model's performance, we consider the following parameterized linear diffusion equation

$$\begin{cases} -\nabla \cdot (\kappa_\mu(\mathbf{x}) \nabla u_\mu(\mathbf{x})) = 1, & \mathbf{x} \in \Omega, \\ u_\mu(\mathbf{x}) = 0, & \mathbf{x} \in \partial\Omega, \end{cases} \quad (13)$$

where $\kappa_\mu(\mathbf{x}) = \exp(-\mu(x_1^2 + x_2^2))$ and the parameter μ varies within $[0, 1]$. We still use the vertex-centered FD scheme Eq. (2) to discretize Eq. (13) on the uniform rectangular cell-centered mesh $\mathcal{M}_c$ with spatial size $N_x = N_y = 256$ and $h_x = h_y = 2/256$. The same U-Net backbone and experiment settings as in the previous ablation study are employed here. The dataset consisting of 10,000 uniformly sampled μ from the interval $[0, 1]$ is used to train the U-Net for 150 epochs. The batch size is set to 5 indicating that the iteration steps is equal to $10,000/5 = 2000$, which is different with the previous ablation studies operated on the parameter-free Laplace equation.

To evaluate the model performance, we uniformly sample 1,000 new parameters to construct a testing dataset. Table 3 reports the mean L_2 errors between the predictions and reference solutions provided. It is evident that both the incomplete solve loss functions $\mathcal{L}_{JK}$ and $\mathcal{L}_{DK}$ can effectively train the U-Net to solve this parameterized problem for all values of $K = 1, 5, 25$. This conclusion is supported by the contours in Fig. 6, where predictions and reference solutions align perfectly. Additionally, Table 3 reveals that the performance of the deep convolutional surrogate model benefits from a larger dataset N_μ and iteration number K . These numerical results clearly demonstrate that both $\mathcal{L}_{JK}$ and $\mathcal{L}_{DK}$ effectively train the U-Net to solve both the Poisson equation and the parameterized diffusion equation. The findings underscore the flexibility and versatility of our approach.

4.2.2. Effect of mesh resolution

To investigate the effect of mesh resolutions on the proposed deep convolutional surrogate model for solving parameterized PDEs, we consider a parameterized Laplace equation with the following analytical solution:

$$u_\mu(x, y) = x(1-x)y(1-y)\exp(\mu(x^2 + y^2)),$$

where $(x, y) \in \Omega = (0, 1)^2$, and the parameter μ ranges from 0.5 to 1.5. Based on this analytical solution, the source term and boundary conditions can be determined accordingly, and the discretized source term is used as the input to the deep convolutional surrogate

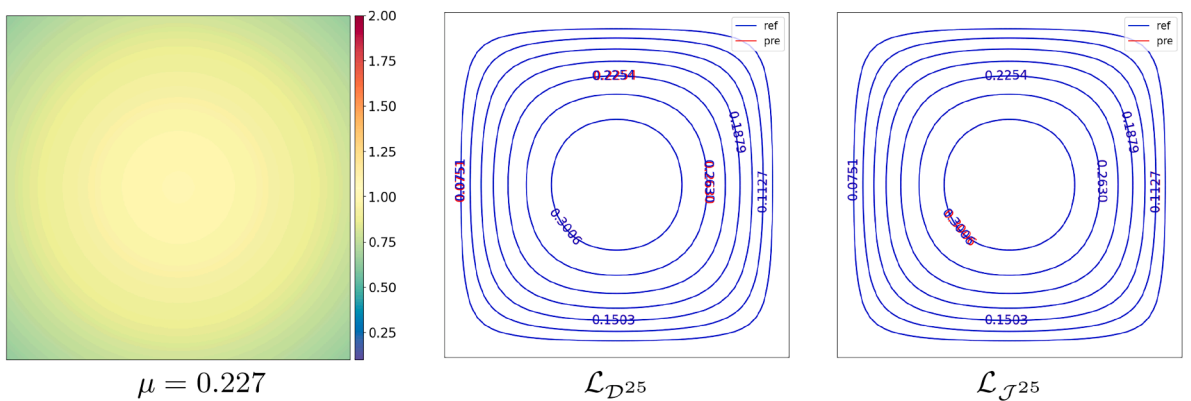


Fig. 6. The contours of the reference solution in the case of $\mu = 0.227$ and the prediction from the deep convolutional surrogate model for solving 2D parameterized diffusion equation, trained using $\mathcal{L}_{J^{25}}$ and $\mathcal{L}_{D^{25}}$, respectively, with $N_\mu = 5000$ training samples.

Table 4

The mean L_2 errors of the deep convolutional surrogate models with the same larger network architecture for solving 2D parameterized Laplace equations under different mesh resolutions.

$N_x = N_y$	L_2	Order
32	1.60e-4	-
64	4.89e-5	1.72
128	2.37e-5	1.03
256	1.37e-5	0.80

model. The models are independently trained on meshes with different resolutions ($N_x = N_y = 32, 64, 128, 256$), all using the same larger-capacity network architecture, U-Net(8,5,(2,2,4,4,6)), to ensure sufficient expressive power for learning the solutions of parameterized PDEs. For each resolution, 15,000 samples are generated for training and 1500 new samples are reserved for testing. The loss function $\mathcal{L}_{J5}$ is adopted, and the training is conducted over 150 epochs with a batch size of 5 and a learning rate of 0.01. The overall training procedure follows the same setup as in the previous section. The numerical results are summarized in Table 4, from which we can see that as the mesh is gradually refined, the proposed deep convolutional surrogate model exhibits a convergence trend. Although its convergence order does not yet match that of traditional numerical scheme (a second order scheme is used in this study), it suggests that using higher-resolution meshes may offere potentially advantages when solving parameterized PDEs.

4.3. 3D Parameterized poisson equation

To further evaluate the scalability of the proposed deep convolutional surrogate model for more complex problems, we consider a 3D parameterized Poisson equation with the following analytical solution:

$$u_\mu(x, y, z) = x(1-x)y(1-y)z(1-z)\exp(\mu(x^2 + y^2 + z^2)),$$

where $(x, y, z) \in \Omega = (0, 1)^3$ and the parameter μ varies within the range $[0.5, 1.5]$. Based on this analytical solution, the corresponding source term and boundary conditions can be directly derived, and the discretized form of the source term is used as input to the deep convolutional surrogate model. Due to computational resource limitations, all experiments are conducted on a uniform grid with a resolution of 64^3 . To apply the proposed surrogate model to the 3D parameterized Poisson equation, the original U-Net architecture-initially designed for 2D problems-must be appropriately adapted. The most straightforward and efficient approach is to replace all 2D convolutional layers with 3D counterparts, while keeping the rest of the network architecture unchanged. The network architecture employed in this study is U-Net(8, 4, (2,2,2,2)). Additionally, no further modification is necessary for the incomplete solve loss function $\mathcal{L}_{SK}$. It is sufficient to use the corresponding 3D PDE solver (we utilize the finite difference method) to generate the appropriate stiffness matrices and load vectors. The model is trained with $\mathcal{L}_{J5}$ on a dataset comprising 10,000 parameter samples uniformly drawn from $[0.5, 1.5]$ and tested on an additional 1500 unseen samples. The corresponding numerical results are presented in Figs. 7 and 8. The histogram in Fig. 7 shows that the deep convolutional surrogate model maintains a very small error level, approximately 10^{-4} , for the vast majority of test parameters. Additionally, we present the true solution, predicted solution, and the absolute error between them on three orthogonal cross-sections in Fig. 8 for a randomly selected parameter, where the extremely small errors further validate the high accuracy of our method.

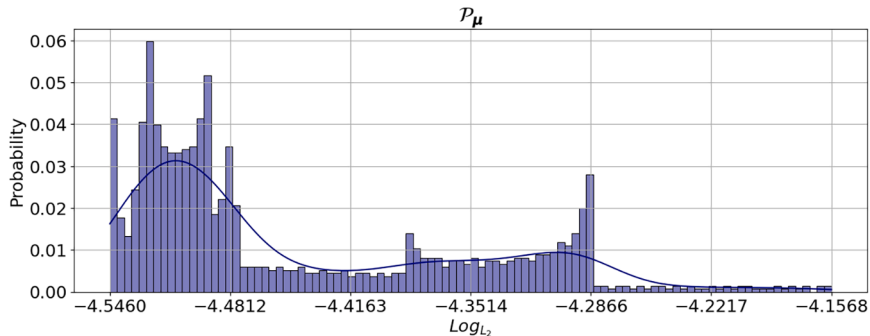


Fig. 7. Histogram of the L_2 solution errors of the 1,500 testing samples for the proposed deep convolutional surrogate model.

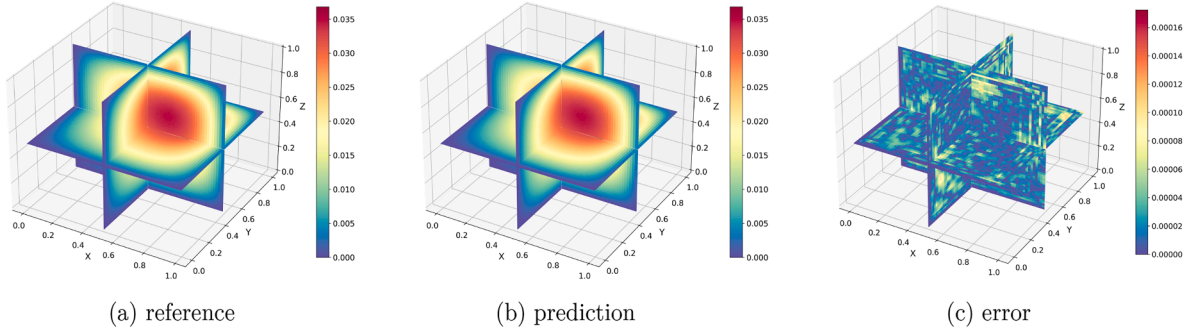


Fig. 8. A sample numerical result for $\mu = 0.896$. (a) the reference solution, (b) the predicted solution, and (c) the corresponding absolute error.

5. Some engineering applications

In this section, we explore three engineering applications of the proposed deep convolutional surrogate model to demonstrate its performance: 1) A heat dissipation problem with parameterized source term and various boundary conditions, used to predict the temperature field for thermal management in electronic system design; 2) a thermal block engineering problem modeled by the 2D diffusion equation with parameterized diffusion coefficients, and 3) a simplified model for immiscible two-phase flow with parameterized source term and viscosity coefficient. In all three applications, the U-Net(8, 5, (2, 4, 4, 6, 8)) is used as the backbone of our deep convolutional surrogate model, with the following settings: the Jacobi iterative method with a maximum of $K = 5$ iterations is employed to construct the loss function; the Adam optimizer is used to train the deep convolutional surrogate model with an initial learning rate set to 10^{-3} , and an exponential learning rate decay strategy with a decay rate of $\gamma = 0.95$; during training, a batch size of 5 is utilized over 150 epochs.

5.1. A heat dissipation problem for HSL-TFP

Heat source layout optimization (HSLO) is an effective technique in satellite layout design for reducing the maximum temperature and improving the heat management of the entire system. Among these techniques, temperature field prediction of heat source layout (HSL-TFP) is the most critical problem, which involves solving heat equation under various boundary conditions. Parameters appear in the source terms are determined by various layouts of heat sources. Chen et al. [27] proposed a deep convolutional surrogate model to address similar challenge, enabling real-time prediction of temperature fields based on heat source layouts. We will extend this approach to tackle more complex problems based on our proposed method.

5.1.1. Problem setting and parameter embedding

The heat dissipation problem for HSL-TFP can be characterized by the Poisson equation

$$-\Delta u_{\mu}(\mathbf{x}) = f_{\mu}(\mathbf{x}),$$

which is defined in a square domain $\Omega = (0, 0.1) \times (0, 0.1)$, and the source term f_{μ} is determined by different heat source layouts, i.e.,

$$f_{\mu}(\mathbf{x}) = \sum_{m=1}^M Q_m \mathbb{1}_{\Omega_m}(\mathbf{x}), \quad (14)$$

where $\{Q_m\}_{m=1}^M$ are the parameters used to control the heat intensity of the m -th block ϕ_m , and $\{\mathbb{1}_{\Omega_m}\}_{m=1}^M$ are piecewise constant functions supported in the rectangular sub-domains $\{\Omega_m\}_{m=1}^M$, which is covered by the m -th electronic component. The same rectangular electronic components as those provided in heat source layout dataset (HSLD) [27] are listed in Table 5. All possible layouts generated from arranging these components form a parameter space denoted by $\mathcal{P}_f$. We use the cell-centered FV scheme Eq. (3) to discretize this problem on a cell-centered uniform rectangular mesh $\mathcal{M}_c$.

Table 5

All rectangular electronic components provided in heat source layout dataset [27].

ID	L (m)	W (m)	Q (W/m^2)	ID	L (m)	W (m)	Q (W/m^2)
1	0.016	0.012	4000	2	0.012	0.006	16,000
3	0.018	0.009	6000	4	0.018	0.011	8000
5	0.018	0.018	10,000	6	0.012	0.012	14,000
7	0.018	0.006	16,000	8	0.009	0.009	20,000
9	0.006	0.024	8000	10	0.006	0.012	16,000
11	0.012	0.024	10,000	12	0.024	0.024	20,000

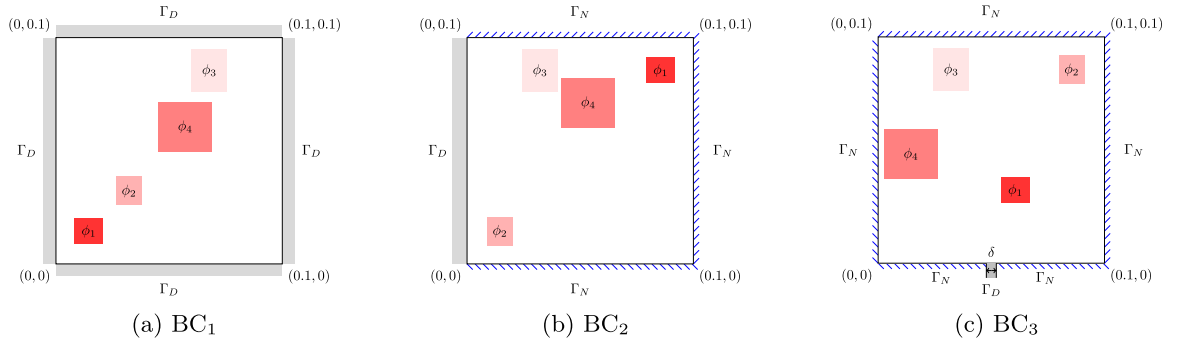


Fig. 9. Three distinctly different of boundary conditions are depicted for the HSL-TFP problem. For simplicity, only four rectangular electronic components are shown. The blue hashed lines represent homogeneous Neumann boundaries, while the thick gray line represents the inhomogeneous Dirichlet boundary with a value of 298. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Furthermore, three distinctly different boundary conditions are considered, as shown in Fig. 9, which includes

1. The whole boundary $\partial\Omega$ is isothermal with constant temperature 298K, denoted by BC_1 . Namely, $g_D = 298$ K on $\Gamma_D = \partial\Omega$.
2. The left side is isothermal with constant temperature 298K, and the others are heat insulated, denoted by BC_2 . That is to say, $g_D = 298$ K on $\Gamma_D = \{0\} \times [0, 0.1]$ and $g_N = 0$ on $\Gamma_N = \partial\Omega \setminus \Gamma_D$.
3. A heat sink with width $\delta = 0.001$ and constant temperature 298K is located on the bottom side of Ω , i.e., $g_D = 298$ K on $\Gamma_D = [0.0495, 0.0505] \times \{0\}$. All the others are heat insulated, i.e., $g_N = 0$ on $\Gamma_N = \partial\Omega \setminus \Gamma_D$. This boundary condition is denoted by BC_3 .

The set containing all three boundary conditions are denoted by $\mathcal{P}_{\text{BC}}$, along with $\mathcal{P}_f$, forms the parameter space $\mathcal{P}_\mu = \mathcal{P}_{\text{BC}} \times \mathcal{P}_f$, where the parameters μ is not only determined by the number of layouts M , their shapes (length L and width W) and intensities Q , but also the three types of boundary conditions.

After sampling a set of μ , the next step is to embed it into the image format. Due to the discretization of the original problem using the cell-centered FV scheme on a cell-centered mesh $\mathcal{M}_c$, we first define the heat source f_μ at the cell center. This constitutes the first channel of input for the deep convolutional surrogate model, with a shape of $N_x \times N_y$. The boundary conditions also need to be embedded into the image format. To achieve this, we introduce a vertex-centered mesh $\mathcal{M}_v$ where the Dirichlet boundary values are assigned to corresponding elements, while all other elements are set to zero. This constitutes the second channel of input, which remains the same size $N_x \times N_y$ as the image format of f_μ . Note that there also could be other methods for embedding the μ into a image form, as long as there is a one to one relation between μ and image formats. The spatial size of $\mathcal{M}_v$ and $\mathcal{M}_c$ is set to $N_x = N_y = 128$ for following experiments.

5.1.2. Performance evaluation

The sequence layout sampling method (SLSM) introduced in [27] is employed to randomly generate 10,000 layouts. To assess the model's performance quantitatively, we test each boundary condition with 2,000 new layouts (totaling 6,000 layouts).

Some sample temperature field prediction results (from the test dataset) under three boundary conditions are shown in Fig. 10. It is easy to observe that the model's predictions closely match with the reference solutions, with errors consistently kept at a low level. These figures illustrate the model's effectiveness in handling various boundary conditions and different layouts, thereby substantiating that the proposed deep convolutional surrogate model has the ability to address parameterized boundary conditions and heat sources through a single training session. The mean L_2 solution errors of all 6,000 layouts and 2,000 layouts for each boundary condition are summarized in Table 6. Meanwhile, histograms of the L_2 errors are also provided in Fig. 11. We observe that for the histogram of all BCs ($\mathcal{P}_{\text{BC}}$), most errors are minimal and there are a few local peaks. Furthermore the errors of the first boundary condition BC_1 are at a relatively lower level comparing to the other two boundary conditions BC_2 and BC_3 .

5.1.3. Performance improvement

To further improve the predictive accuracy of the deep convolutional surrogate model, we adopt the strategy proposed by Zhao et al. [19], incorporating physical prior knowledge into the modeling process. Specifically, according to the minimum principle of the steady-state heat conduction equation, when the domain contains a non-negative internal heat source (i.e., the source term $f_\mu(x) \geq 0$) and the Dirichlet boundary condition is constant g_D , the temperature within the domain satisfies:

$$u(x) \geq g_D, \quad \forall x \in \Omega.$$

Based on this physical constraint, we redesigned the model output to predict the deviation of the temperature field from g_D , rather than predicting the temperature field directly:

$$u_\mu(\Theta) = g_D + \tilde{u}_\mu(\Theta).$$

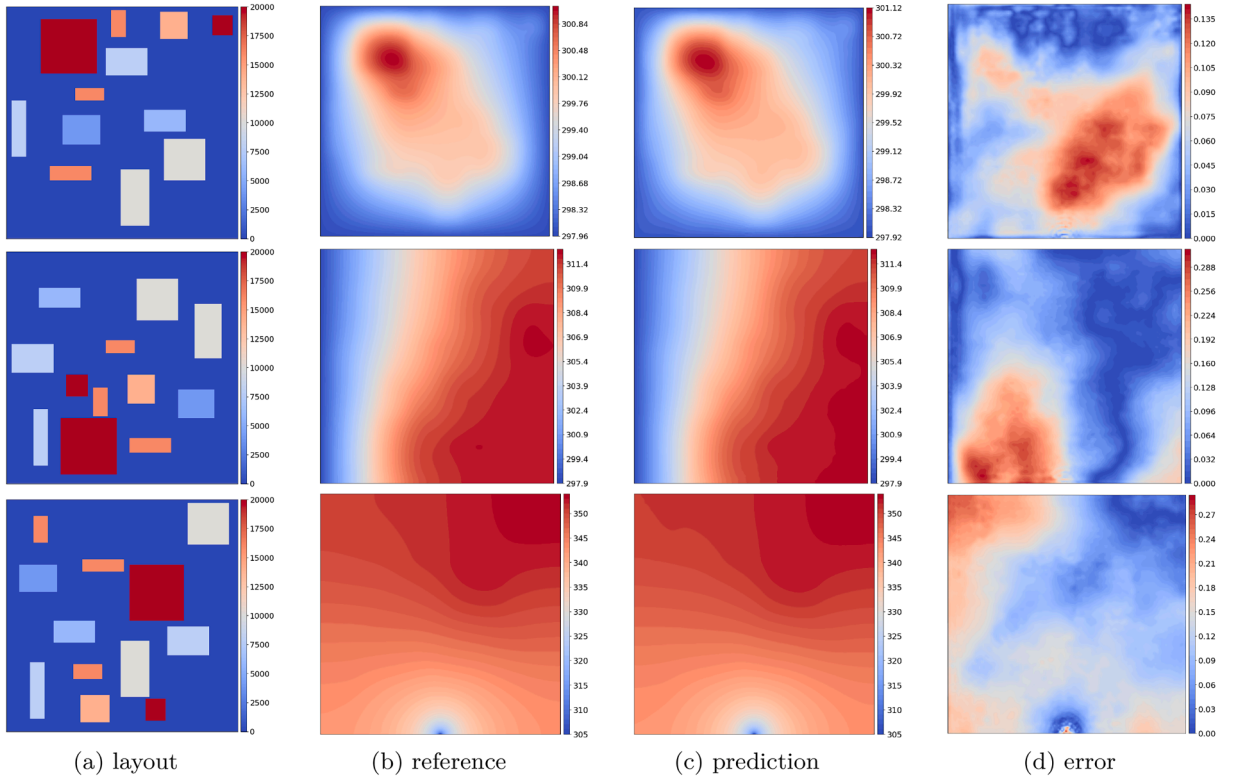


Fig. 10. A sample numerical result under BC_1 (top row), BC_2 (middle row), and BC_3 (bottom row) for the HSL-TFP problem. (a) The heat source layout, (b) the reference solution, (c) the predicted solution, and (d) the corresponding absolute error.

Here, the output of the network $\tilde{u}_\mu(\Theta)$ is required to be nonnegative. To enforce this non-negativity constraint, we employ a squared function as the activation function at the network's output layer. All other settings, including the dataset, loss function, network architecture, and training strategy, remain unchanged. This approach demonstrates significant improvement in predictive accuracy, as evidenced by the numerical results in Table 6, where the performance with and without the incorporation of physical priors is compared. We also plotted the corresponding error histograms in Fig. 12 to further demonstrate the impact of incorporating physical priors on model performance. Compared to the histograms without physical priors in Fig. 11, the model with priors shows a clear improvement in overall prediction accuracy, with the most significant enhancement observed in the BC_1 case.

5.2. Thermal block engineering problem

For optimal control problems in engineering applications, the diffusion coefficients κ_μ are often crucial parameters. Here we use a thermal block engineering problem to test the performance of the proposed deep convolutional surrogate model for such purpose.

5.2.1. Problem setting and parameter embedding

The thermal block engineering problem is described by the parameterized linear diffusion equation

$$-\nabla \cdot (\kappa_\mu(\mathbf{x}) \nabla u_\mu(\mathbf{x})) = 0,$$

Table 6

Results on the mean L_2 solution errors for the proposed deep convolutional surrogate model with/without using the physical priors, 2,000 layouts for each boundary condition $\{BC_i\}_{i=1}^3$, and totaling 6000 layouts $BC = \cup_{i=1}^3 BC_i$ are used for testing.

	BC	BC_1	BC_2	BC_3
w/o prior	3.46e-2	2.00e-2	2.37e-2	6.01e-2
w prior	1.90e-2	1.66e-3	1.01e-2	4.49e-2

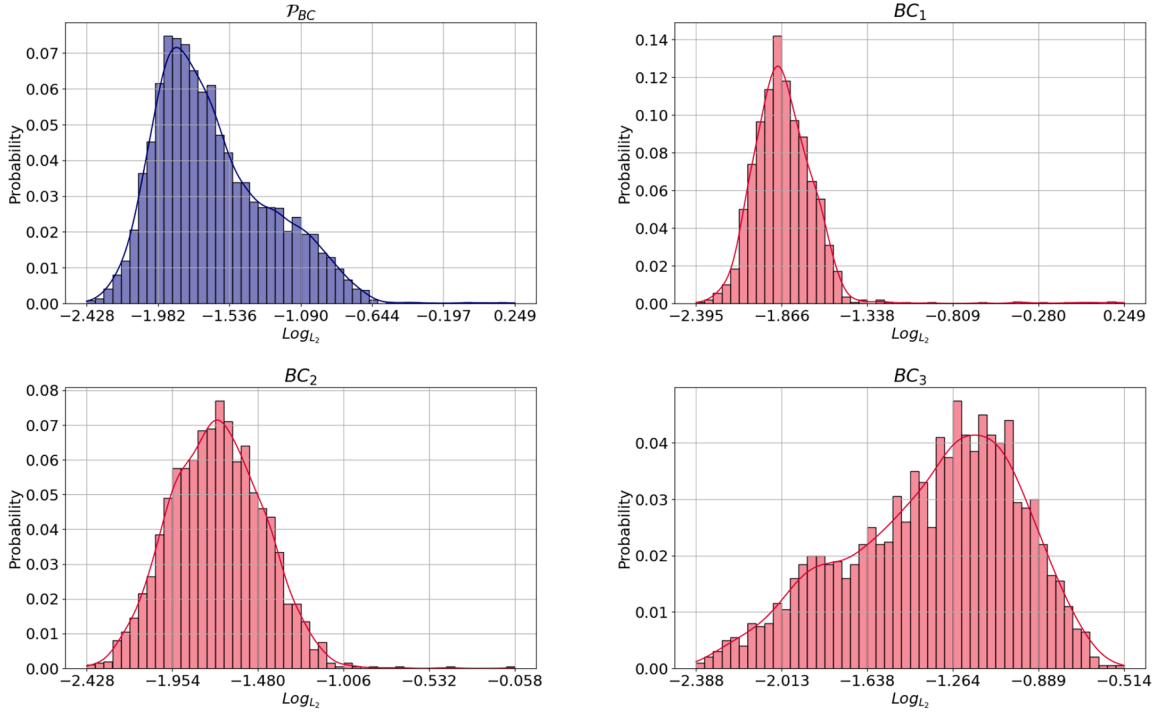


Fig. 11. Histograms of the L_2 solution errors of the 6,000 testing samples for the proposed deep convolutional surrogate model without prior knowledge for the HSL-TFP problem.

which is defined on a square domain $\Omega = (0, 1) \times (0, 1)$. The parameterized diffusion coefficients are defined by piecewise functions $\kappa_\mu(\mathbf{x})$ as follows:

$$\kappa_\mu(\mathbf{x}) = \sum_{i=1}^M \mu_i \mathbb{1}_{\Omega_i}(\mathbf{x}), \quad (15)$$

where $\mu = (\mu_1, \mu_2, \dots, \mu_M) \in [0.1, 10]^M$ with $M = m^2$, $m \in \{2, 3, 4\}$. Here the domain Ω is equally partitioned into $M = m \times m$ sub-blocks $\{\Omega_m\}_{m=1}^M$, as shown in the Fig. 13, and each block has a constant diffusion coefficient $\kappa = \mu_i$. Therefore, the parameter space for the parameterized problem is $\mathcal{P}_\mu = [0.1, 10]^{2^2} \cup [0.1, 10]^{3^2} \cup [0.1, 10]^{4^2}$. The top side of Ω is characterized by a homogeneous Dirichlet boundary: $\Gamma_D = [0, 1] \times \{1\}$ with $g_D = 0$. Conversely, the bottom side features a Neumann boundary condition where a unit flux $g_N = -1$ flows into Ω . The remaining left and right sides are homogeneous Neumann boundaries: $\Gamma_N = [0, 1] \times \{0\} \cup \{0, 1\} \times [0, 1]$. We still use the cell-centered FVM Eq. (3) to discretize this problem on a cell-centered mesh $\mathcal{M}_c$.

The parameter embedding for this problem is straightforward: restrict the diffusion coefficient to the cell centers of the mesh $\mathcal{M}_c$, thereby forming a single-channel input for the deep convolutional surrogate model, with the size of $N_x \times N_y$. The spatial size of $\mathcal{M}_c$ is set to $N_x = N_y = 96$, ensuring that the discontinuities in κ_μ do not cross through the interior of each cell. This is essential for the cell-centered FV scheme to maintain numerical stability and accuracy in simulations.

5.2.2. Performance evaluation

Similar to the previous application problem, the dataset consists of 10,000 μ which are sampled uniformly from $\mathcal{P}_\mu$. In particular, we uniformly sample 10,000 integer numbers m from the set $\{2, 3, 4\}$ in first, then the corresponding diffusion coefficients κ_μ are uniformly sampled from $[0.1, 10]^{m^2}$ for every m . Furthermore, additional 6,000 new κ_μ were randomly selected using the same procedure to build a new dataset for testing, in which 2,000 κ_μ are sampled for each $m = 2, 3, 4$. The numerical solutions provided by the same FV scheme on $\mathcal{M}_c$ are used as reference solutions.

Some sample prediction results (from the test dataset) for various block-wise diffusion coefficients are illustrated in Fig. 14. It is easy to see that the predictions closely match the reference solutions in all cases. The L_2 solution errors are plotted as a histogram and shown in Fig. 15. Based on these statistical results, we observe that the majority of errors are minimal, with only a few instances exhibiting relatively larger errors, as indicated by the extended tails. This underscores the model's consistent performance across different test cases, highlighting its robust generalization ability across all scenarios.

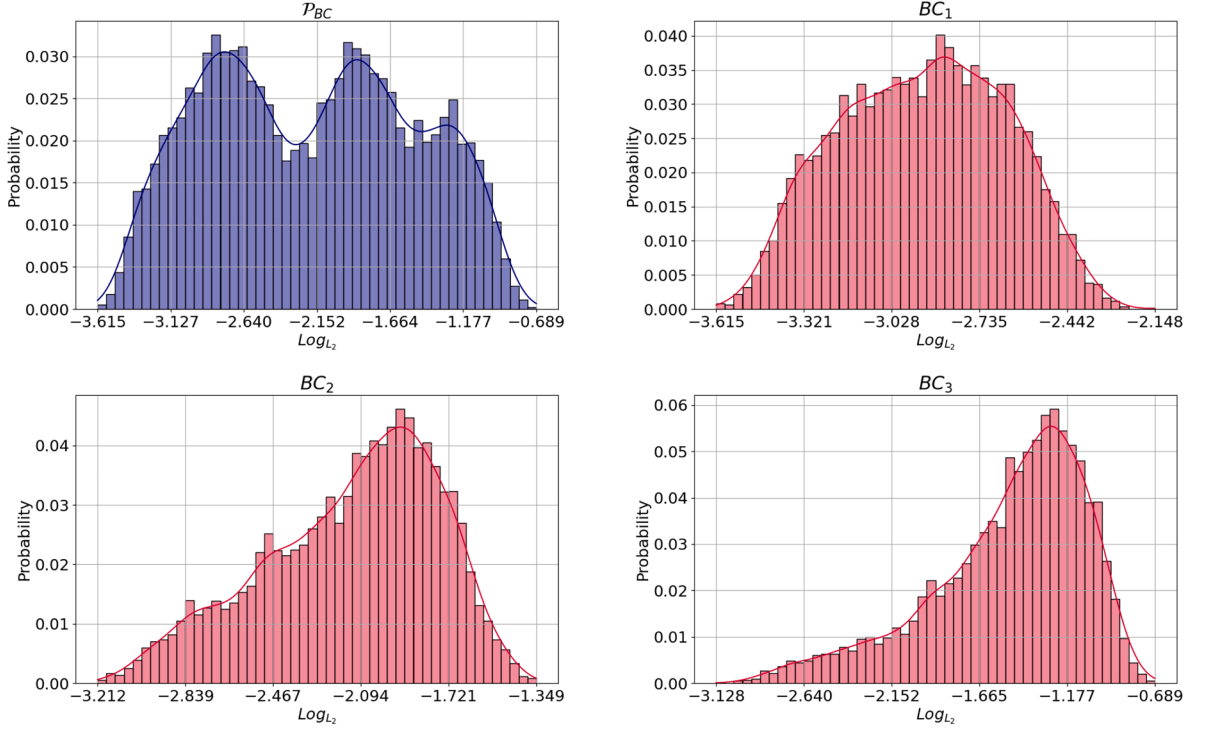


Fig. 12. Histograms of the L_2 solution errors of the 6,000 testing samples for the proposed deep convolutional surrogate model with prior knowledge for the HSL-TFP problem.

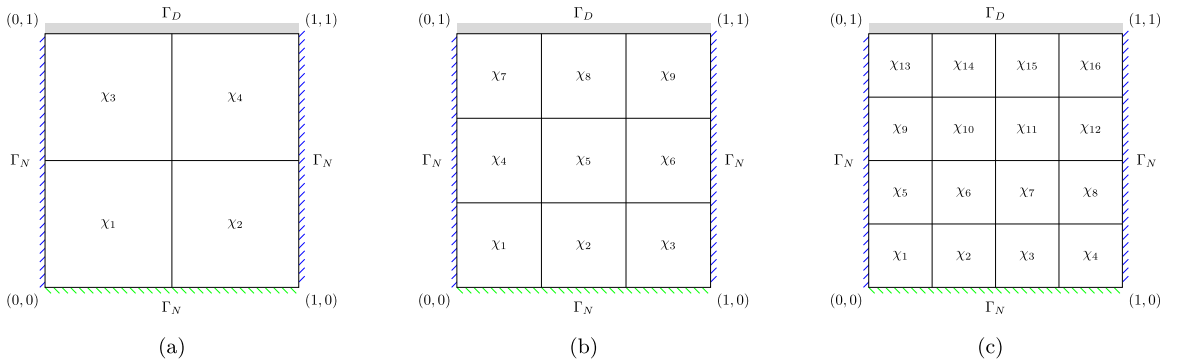


Fig. 13. Three settings of the parameterized diffusive coefficient problem are provided, each share the same boundary conditions: a green line at the bottom signifies a Neumann boundary with unit flux, blue lines on the left and right denote homogeneous Neumann boundaries, and a thick gray line at the top indicates a homogeneous Dirichlet boundary. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

5.3. A simplified model for immiscible two-phase flow

In this subsection, we study a simplified nonlinear diffusion equation that describes immiscible two-phase flow in porous media. This model is inspired by the work in [28] and further explored by [29].

5.3.1. Problem setting and parameter embedding

The model of immiscible two-phase flow can be described by the nonlinear parameterized diffusion equation

$$-\nabla \cdot (\kappa_\mu(u_\mu(\mathbf{x})) \nabla u_\mu(\mathbf{x})) = f_v(\mathbf{x})$$

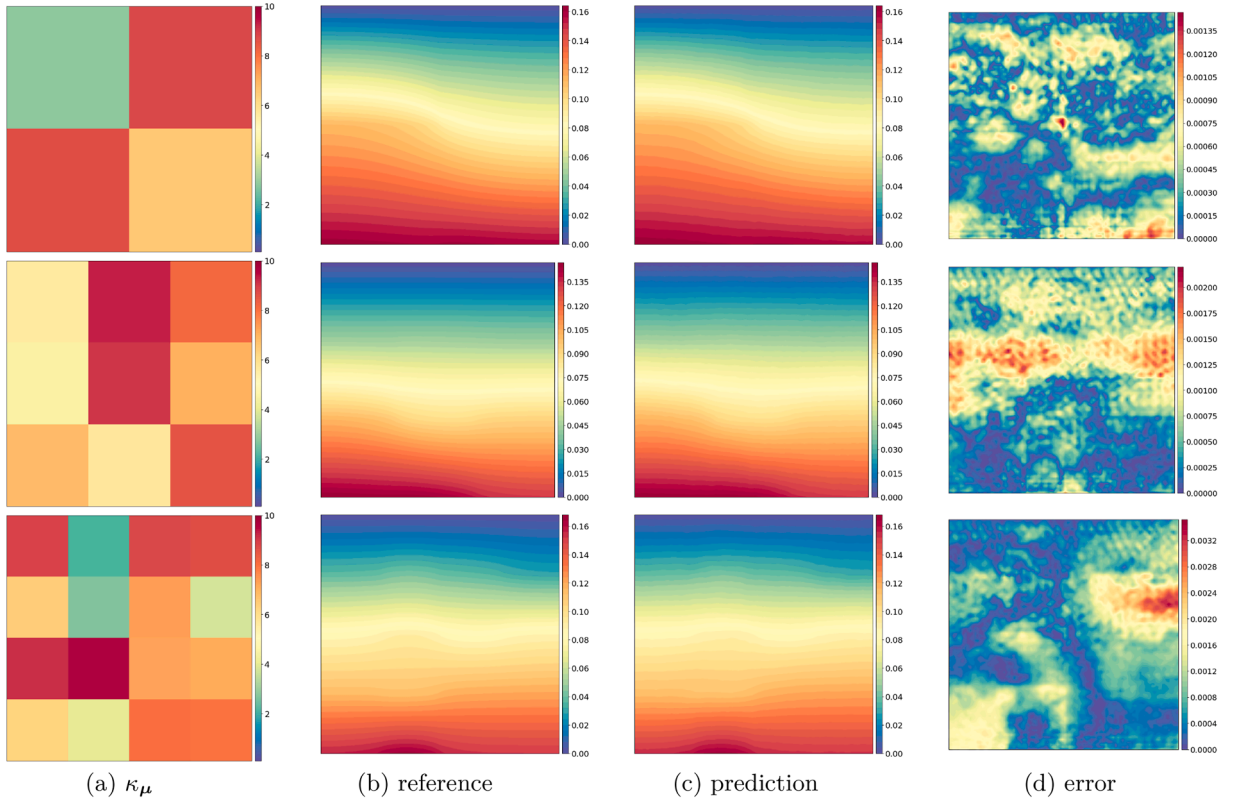


Fig. 14. A sample numerical result in the case of $M = 2^2$ (top row), $M = 3^2$ (middle row), and $M = 4^2$ (bottom row) blocks for the thermal block engineering problem. (a) The diffusion coefficient, (b) the reference solution, (c) the predicted solution, and (d) the corresponding absolute error.

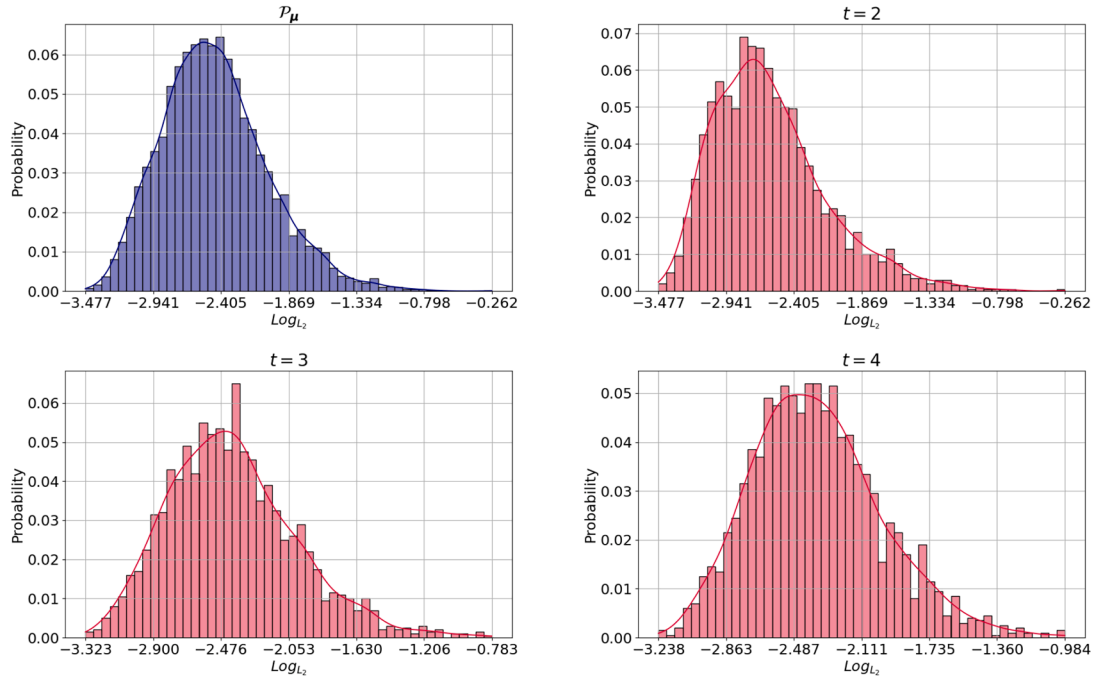


Fig. 15. Histograms of the L_2 solution errors of the 6000 testing samples for the proposed deep convolutional surrogate model for the thermal block engineering problem.

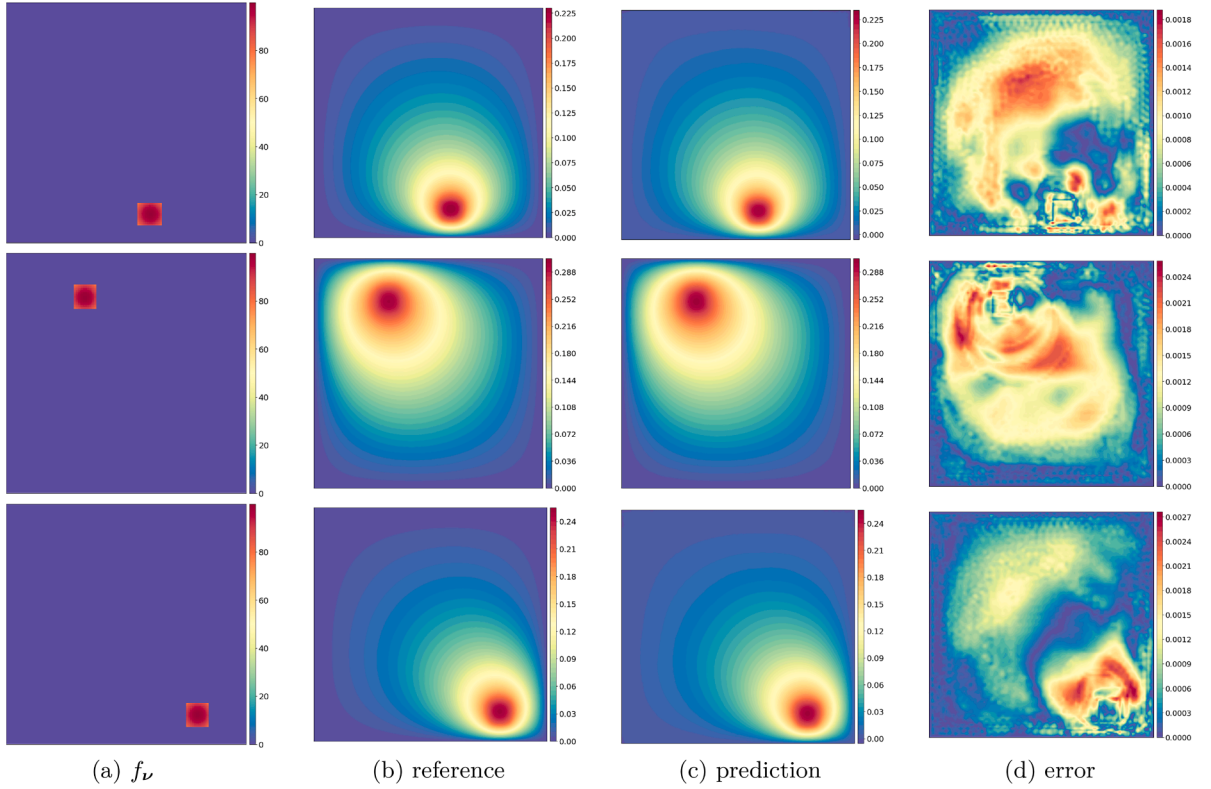


Fig. 16. A sample numerical result for $\mu = 0.857$ (top row), $\mu = 0.423$ (middle row), and $\mu = 0.627$ (bottom row) for the immiscible two-phase flow problem. (a) f_v , (b) the reference solution, (c) the predicted solution, and (d) the corresponding absolute error.

with homogeneous Dirichlet boundary condition defined on a square domain $\Omega = (0, 1) \times (0, 1)$. Both the nonlinear diffusion coefficient and source term are parameterized and their specific forms are given by

$$\kappa_\mu(u) = \left(\frac{3u(1-u)}{3u^3 + (1-u)^3} \right)^2 + \mu \quad (16)$$

and

$$f_v(\mathbf{x}) = 100 \exp(-50 \|\mathbf{x} - \mathbf{v}\|_2^2) \mathbb{1}_{\Omega_v}(\mathbf{x}), \quad (17)$$

where the parameter $\mathbf{v} = (v_1, v_2)^T \in \Omega$ is the center of a square subregion $\Omega_v = [v_1 - H/2, v_1 + H/2] \times [v_2 - H/2, v_2 + H/2]$ with a side length $H = 0.05$. The parameterized sources f_v are locally supported continuous functions with different centers $\mathbf{v}$. Thus it results in the parameter space $\mathcal{P}_\mu = (\mu, \mathbf{v}) = [0.1, 1] \times [0.1, 0.9]^2$. We employ the vertex-centered FDM to discretize the equation with the vertex-centered mesh $\mathcal{M}_v$. We perform parameter embedding as follows: Firstly, embedding μ is straightforward; its value is multiplied by an $N_x \times N_y$ all-ones matrix, forming the first channel of the deep convolutional surrogate model input. Next, we evaluate f_v on the grid nodes of $\mathcal{M}_v$, creating the second channel of input for the deep convolutional surrogate model. The spatial size of the vertex-centered mesh $\mathcal{M}_v$ is set to $N_x = N_y = 128$.

5.3.2. Performance evaluation

A total of 10,000 parameters are sampled uniformly from $\mathcal{P}_\mu$ for training. To demonstrate the generalization performance of the model, we test and evaluate the quantitative results from 2,000 newly sampled μ , the numerical solutions provided by the FD scheme on the same vertex-centered mesh $\mathcal{M}_v$ is treated as reference solutions. During the training process, the maximum steps and threshold of Picard's iteration in Algorithm 3 are set to $P_{\max} = 100$, $\epsilon^p = 1 \times 10^{-7}$, respectively.

Some sampled prediction results (from the test dataset) for the immiscible two-phase flow are illustrated in Fig. 16. It is evident that the predicted solutions closely approximate the reference solutions across various scenarios, demonstrating minimal differences. This indicates that the deep convolutional surrogate model is capable of effectively learning solutions to nonlinear diffusion problems under different parameter conditions. The contour plots in Fig. 17 (a)-(c) illustrate the nearly complete overlap between predicted and reference solutions across these three scenarios, further demonstrating the prediction accuracy of the deep convolutional surrogate model. The histogram of L_2 errors for the 2000 test data are presented in Fig. 18. We observe that in most cases the model's prediction accuracy is very good. Even in a few instances where prediction errors are slightly larger, they still remain within acceptable ranges.

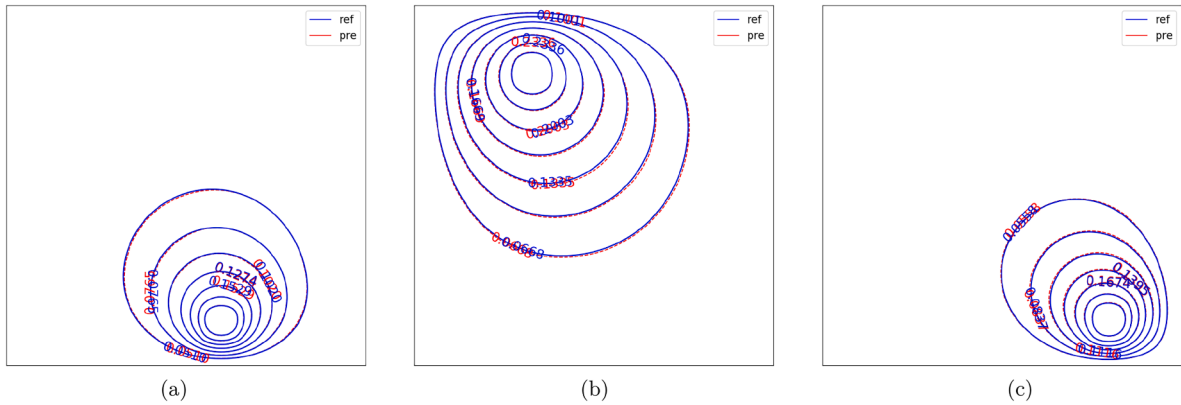


Fig. 17. The corresponding contours plots of the predictions for the three test cases presented in Fig. 16, respectively.

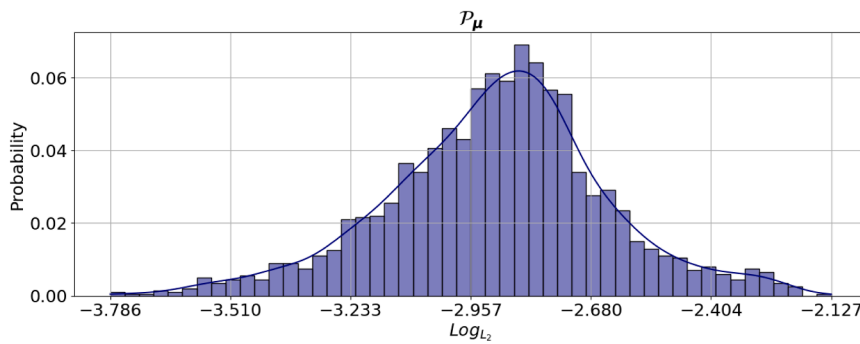


Fig. 18. Histogram of the L_2 solution errors of the 2,000 testing samples for the proposed deep convolutional surrogate model for the immiscible two-phase flow problem.

6. Conclusion

In this paper, we have proposed a novel deep convolutional surrogate model in the end-to-end fashion, which integrates the convolutional neural network approach with traditional numerical methods to learn parameterized steady-state diffusion problems. Specifically, we embed the parameters into images and adopt a slightly modified U-Net as the backbone of our model. The proposed incomplete solve loss function is constructed by combining conventional numerical discretizations and popular linear iterative solvers. Our training strategy for nonlinear problems is formulated by utilizing that for linear problems and the Picard's iteration. Ablation studies demonstrate that the proposed loss function is capable of efficiently and effectively training the model. Extensive experimental results for three engineering application problems further demonstrate the versatility and robustness of our deep convolutional surrogate model. In conclusion, we not only design a deep convolutional surrogate model with promising ability for solving parameterized diffusion problems, but also build a framework for utilizing incomplete iterative solvers in the loss function and training process for deep learning. In the future, we will continue extending this method to more problems in scientific and engineering applications, which can provide new vitality in the intersection field of applied mathematics and machine learning.

CRedit authorship contribution statement

Junqing Jia: Writing – original draft, Software, Methodology; **Xiaoping Zhang:** Writing – review & editing, Writing – original draft, Supervision, Methodology; **Shuting Wang:** Writing – original draft, Software; **Lili Ju:** Writing – review & editing, Supervision, Methodology; **Hui Feng:** Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

X. Zhang's research is partially supported by Hubei Provincial Natural Science Foundation of China under grant number 2024AFB749 and the National Key Research and Development Program of China under grant number 2021YFD1900805-02. L. Ju's research is partially supported by US Department of Energy under grant number DE-SC0022254.

References

- [1] Z. Li, N.B. Kovachki, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, et al., Fourier neural operator for parameterized partial differential equations, in: International Conference on Learning Representations, 2021.
- [2] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nature Machine Intelligence* 3 (2021) 218–229.
- [3] N.R. Franco, S. Fresca, F. Tombari, A. Manzoni, Deep learning-based surrogate models for parametrized PDEs: handling geometric variability through graph neural networks, *Chaos* 33 (2023) 123121.
- [4] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [5] J.A. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *J. Comput. Phys.* 375 (2018) 1339–1364.
- [6] B. Yu, et al., The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems, *Commun. Math. Stat.* 6 (1) (2018) 1–12.
- [7] G. Pang, L. Lu, G.E. Karniadakis, FPINNs: fractional physics-informed neural networks, *SIAM J. Sci. Comput.* 41 (4) (2019) A2603–A2626.
- [8] L. Yang, D. Zhang, G.E. Karniadakis, Physics-informed generative adversarial networks for stochastic differential equations, *SIAM J. Sci. Comput.* 42 (1) (2020) A292–A317.
- [9] D. Zhang, L. Lu, L. Guo, G.E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, *J. Comput. Phys.* 39 (2019) 108850.
- [10] D. Zhang, L. Guo, G.E. Karniadakis, Learning in modal space: solving time-dependent stochastic PDEs using physics-informed neural networks, *J. Comput. Phys.* 42 (2) (2020) A639–A665.
- [11] W. E, B. Yu, The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems, *Commun. Math. Stat.* 6 (1) (2018) 1–12.
- [12] Y. Liao, P. Ming, Deep Nitsche method: deep Ritz method with essential boundary conditions, *Commun. Comput. Phys.* 29 (5) (2021) 1365–1384. <https://doi.org/10.4208/cicp.oa-2020-0219>
- [13] J. Müller, M. Zeinhofer, Deep Ritz revisited, in: DeepDiffEq Workshop at International Conference on Learning Representations, 2020.
- [14] Z. Wang, Z. Zhang, A mesh-free method for interface problems using the deep learning approach, *J. Comput. Phys.* 400 (2020) 108963. <https://doi.org/10.1016/j.jcp.2019.108963>
- [15] J. Chen, R. Du, K. Wu, A comparison study of deep Galerkin method and deep Ritz method for elliptic problems with different boundary conditions, *Commun. Math. Res.* 36 (3) (2020) 354–376.
- [16] A. Kovacs, L. Exl, A. Kornell, J. Fischbacher, M. Hovorka, M. Gusenbauer, L. Breth, H. Oezelt, M. Yano, N. Sakuma, et al., Conditional physics informed neural networks, *Commun. Nonlinear Sci. Numer. Simul.* 104 (2022) 106041.
- [17] X. Zhang, Y. Zhu, J. Wang, L. Ju, Y. Qian, M. Ye, J. Yang, GW-PINN: A deep learning algorithm for solving groundwater flow equations, *Adv. Water Resour.* 165 (2022) 104243.
- [18] J.N. Fuhg, A. Karmarkar, T. Kadeethum, H. Yoon, N. Bouklas, Deep convolutional Ritz method: parameterized PDE surrogates without labeled data, *Appl. Math. Mech.* 44 (7) (2023) 1151–1174.
- [19] X. Zhao, Z. Gong, Y. Zhang, W. Yao, X. Chen, Physics-informed convolutional neural networks for temperature field prediction of heat source layout without labeled data, *Eng. Appl. Artif. Intell.* 117 (2023) 105516.
- [20] Y. Zhang, Z. Gong, W. Zhou, X. Zhao, X. Zheng, W. Yao, Multi-fidelity surrogate modeling for temperature field prediction using deep convolution neural network, *Eng. Appl. Artif. Intell.* 123 (2023) 106354.
- [21] H. Gao, L. Sun, J.-X. Wang, Phygeonet: physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state PDEs on irregular domain, *J. Comput. Phys.* 428 (2021) 110079.
- [22] Y. Geng, Y. Teng, Z. Wang, L. Ju, A deep learning method for the dynamics of classic and conservative Allen-Cahn equations based on fully-discrete operators, *J. Comput. Phys.* 496 (2024) 112589.
- [23] S. Miao, J. Wu, Y. Yao, An interpolation-free cell-centered discretization of the heterogeneous and anisotropic diffusion problems on polygonal meshes, *Comp. Math. Appl.* 130 (2023) 105–118.
- [24] G.H. Golub, C.F. Van Loan, *Matrix Computations - 4th Edition*, Johns Hopkins University Press, Philadelphia, PA, Philadelphia, PA, 2013. <https://doi.org/10.1137/1.9781421407944>
- [25] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
- [26] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Medical Image Computing and computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18, Springer, 2015, pp. 234–241.
- [27] X. Chen, X. Zhao, Z. Gong, J. Zhang, W. Zhou, X. Chen, W. Yao, A deep neural network surrogate modeling benchmark for temperature field prediction of heat source layout, *Sci. China Phys. Mech. Astron.* 64 (11) (2021) 1.
- [28] A. Michel, A finite volume scheme for two-phase immiscible flow in porous media, *SIAM J. Numer. Anal.* 41 (4) (2003) 1301–1317.
- [29] K. Smetana, M. Ohlberger, Hierarchical model reduction of nonlinear partial differential equations based on the adaptive empirical projection method and reduced basis techniques, *ESAIM Math. Modell. Numer. Anal.* 51 (2) (2017) 641–677.